

UNIVERSITÉ HASSAN II DE CASABLANCA
FACULTÉ DES SCIENCES BEN M'SIK
Master Intelligence Artificielle

MLSecOps Platform

End-to-End Secure Machine Learning Operations

Rapport de Projet de Fin de Module

Réalisé par : Étudiant 1 – *Benjelloun Abderrahmane*
Étudiant 2 – *Soumia Zahir*
Étudiant 3 – *Ayman Sabiri*
Étudiant 4 – *Hamza Rbib*
Étudiant 5 – *Oumaima El Habti*
Étudiant 6 – *Abderrahmane Hassani*
Étudiant 7 – *Houssame Bouyous*
Étudiant 8 – *Marwan Ait-Tamanait*

Encadré par : Pr. *Aitdaoud Mohammed*

Filière : Master IA

Année Universitaire 2025–2026

Résumé

La prolifération des systèmes d'apprentissage automatique (machine learning) dans les domaines critiques pour la sécurité — allant de la détection d'intrusions réseau à la prise de décision autonome — a souligné l'impératif d'intégrer la sécurité directement à chaque étape du cycle de vie du ML. Ce rapport présente la **Plateforme MLSecOps**, un système complet de niveau production qui unifie les opérations de données (*DataOps*), les opérations de ML (*MLOps*) et les opérations de sécurité (*SecOps*) au sein d'une seule architecture entièrement conteneurisée.

La plateforme implémente un pipeline de bout en bout pour la détection d'anomalies réseau basé sur le jeu de données CICIDS2017. Les données de flux réseau brutes sont ingérées en temps réel via un backbone de streaming Kafka/Redpanda, nettoyées et normalisées par un processeur de flux dédié, validées par un framework de qualité de données de 831 lignes imposant des règles d'exhaustivité, de validité et d'intégrité, puis envoyées dans un classificateur XGBoost entraîné avec une validation croisée stratifiée, une pondération d'échantillons atténuée par racine carrée et un suivi complet des expériences via MLflow. Un service d'inférence FastAPI expose des endpoints de prédiction pour les flux uniques et par lots, instrumentés avec des compteurs et histogrammes Prometheus pour une surveillance en direct. L'ensemble du système est orchestré par Dagster, qui gère les actifs de données sous forme de graphe orienté acyclique (DAG) avec des capteurs automatiques, des tâches planifiées et des portes de qualité qui arrêtent le pipeline en cas de problème. Une interface frontend moderne en React + TypeScript offre un tableau de bord pour la visualisation des menaces en temps réel, l'inspection de flux unique, l'analyse par lots via CSV et la surveillance de la santé du système.

La sécurité est intégrée dans toute la plateforme : des hooks de pre-commit imposent le formatage du code et l'analyse statique ; un pipeline CI/CD GitHub Actions exécute le linting, les tests et la construction d'images Docker à chaque push ; les colonnes sensibles et d'identification sont explicitement supprimées avant l'entraînement du modèle pour éviter la fuite de données ; et les images de conteneurs s'exécutent en tant qu'utilisateurs non-root selon les meilleures pratiques de sécurité de Docker. La couche de transformation d'Analytics via dbt fournit des modèles SQL reproductibles et versionnés avec des tests au niveau du schéma sur un entrepôt de données léger DuckDB.

Ce rapport fournit une analyse approfondie, au niveau du code, de chaque composant architectural, explique la logique derrière chaque décision de conception et situe le travail dans le contexte académique et industriel plus large du MLSecOps.

Mots-clés : MLOps, MLSecOps, DataOps, Détection d'anomalies réseau, XGBoost, Dagster, FastAPI, Kafka, Redpanda, Docker, MLflow, Prometheus, Qualité des données, CI/CD, CICIDS2017.

Remerciements

Nous tenons à exprimer notre sincère gratitude à notre encadrant, *Pr. [Nom de l'Encadrant]*, pour ses précieux conseils et son soutien tout au long de ce projet. Nous adressons également nos remerciements au corps professoral et au personnel du Département d'Informatique de la Faculté des Sciences Ben M'Sik, Université Hassan II de Casablanca, pour avoir fourni les bases académiques sur lesquelles ce travail est construit.

Nous sommes également reconnaissants envers la communauté open source dans son ensemble, dont les outils — Dagster, MLflow, FastAPI, Redpanda, dbt, Prometheus, et bien d'autres — ont rendu cette plateforme possible. Des remerciements particuliers vont à l'Institut Canadien pour la Cybersécurité pour avoir mis à disposition publique le jeu de données CICIDS2017, permettant ainsi des recherches reproductibles dans la détection d'intrusions réseau.

Enfin, nous remercions nos camarades de classe du programme Master MLOps & DataOps pour leurs retours constructifs et leur esprit de collaboration.

Contents

Résumé	i
Remerciements	ii
1 Introduction	1
1.1 Contexte et Motivation	1
1.2 Problématique	2
1.3 Objectifs	2
1.4 Structure du Rapport	3
2 Architecture du Système & Technologies	5
2.1 Aperçu de l'Architecture Globale	5
2.2 Analyse Approfondie de la Pile Technologique	8
2.2.1 Python 3.10 — Environnement d'Exécution Principal	8
2.2.2 FastAPI — Framework d'API REST	8
2.2.3 Dagster — Orchestration du Pipeline	8
2.2.4 Redpanda — Backbone de Streaming	9
2.2.5 dbt + DuckDB — Transformation des Données	9
2.2.6 MLflow — Suivi des Expériences & Registre de Modèles	9
2.2.7 Prometheus + Grafana — Surveillance	9
2.2.8 React + TypeScript + Vite — Frontend	10
2.2.9 Docker — Stratégie de Conteneurisation	10
3 DataOps & Ingénierie du Pipeline	12
3.1 Source de Données : Dataset CICIDS2017	12
3.2 Pipeline d'Ingestion des Données	13
3.2.1 Téléchargement Automatisé du Dataset	13
3.2.2 Ingestion en Streaming avec Kafka Producer	13
3.3 Pipeline de Nettoyage de Flux (Stream Cleaning)	15
3.3.1 Logique de Nettoyage des Enregistrements	15
3.3.2 Sondage Inactif et Arrêt Doux (Graceful Shutdown)	16
3.4 Framework de Qualité des Données	16
3.4.1 Architecture	16
3.4.2 Étape 1 : Validation du CSV Brut	16
3.4.3 Étape 2 : Validation des Enregistrements Nettoyés	17
3.4.4 Étape 3 : Validation de la Sortie d'Inférence	17
3.5 Couche de Transformation dbt	18
3.5.1 Configuration du Projet	18

3.5.2	Modèle de Staging	18
3.5.3	Tests au Niveau du Schéma	19
3.6	Préparation des Données pour le ML	19
3.6.1	Conversion de JSONL en Parquet	20
3.6.2	Équilibrage du Dataset	20
4	Entraînement du Modèle & Suivi des Expériences	22
4.1	Ingénierie des Caractéristiques (Feature Engineering)	22
4.1.1	Sélection des Caractéristiques : Le Sous-ensemble de 33 Caractéristiques	22
4.1.2	Suppression des Colonnes Sensibles	24
4.1.3	Extraction des Étiquettes (Labels)	25
4.2	Pipeline d'Entraînement	25
4.2.1	Sanitisation des Données & Prévention des Fuites	25
4.2.2	Pondération des Échantillons pour les Classes d'Attaques Rares	26
4.2.3	Validation Croisée Stratifiée	26
4.2.4	Configuration du Classificateur XGBoost	27
4.2.5	Intégration MLflow	27
4.3	Évaluation du Modèle	28
4.4	Module de Prédiction & d'Inférence	29
4.4.1	Chargement du Modèle avec Repli sur le Registre	29
4.4.2	Prédiction Unique et par Lots (Batch)	29
5	Intégration de la Sécurité (MLSecOps)	31
5.1	Qualité du Code & Analyse Statique	31
5.1.1	Crochets de Pré-Validation (Pre-Commit Hooks)	31
5.1.2	Analyse Lint avec Flake8	32
5.2	Pipeline CI/CD	32
5.3	Stratégie de Test	34
5.3.1	Configuration de Pytest	34
5.3.2	Tests de Qualité des Données (537 Lignes)	34
5.4	Sécurité des Conteneurs	35
5.4.1	Exécution Sans Privilèges Root	35
5.4.2	Images de Base Allégées (Slim)	36
5.4.3	Isolation du Réseau Docker	36
5.5	Prévention des Fuites de Données	36
5.6	Sécurité de l'API	36
6	Orchestration avec Dagster	38
6.1	Conception de Pipeline Centrée sur les Actifs (Assets)	38
6.2	Actifs du Pipeline (Pipeline Assets)	39
6.2.1	Actif d'Ingestion de Flux (Streaming Ingestion Asset)	39
6.2.2	Actif de Nettoyage de Flux (Streaming Cleaning Asset)	39
6.2.3	Actif d'Inférence de Modèle (Model Inference Asset)	40
6.3	Capteurs et Planifications (Sensors and Schedules)	40
6.3.1	Capteur du Système de Fichiers (File-System Sensor)	40
6.3.2	Planification de Sécurité Horaire (Hourly Safety Schedule)	41
6.4	Assemblage des Définitions (Definitions Assembly)	41

7	Déploiement & Surveillance	42
7.1	Orchestration avec Docker Compose	42
7.1.1	Définitions de Services	42
7.1.2	Gestion des Volumes	43
7.1.3	Topologie du Réseau	43
7.2	Couche de Service FastAPI (Serving Layer)	44
7.2.1	Cycle de Vie de l'Application	44
7.2.2	Instrumentation Prometheus	45
7.2.3	Validation de Schéma Pydantic	46
7.3	Application Frontend	47
7.3.1	Architecture	47
7.3.2	Architecture du Client API	48
7.3.3	Stratégie de Traitement par Lots (Batch Processing)	49
7.4	Architecture de Surveillance (Monitoring)	51
8	Gestion de Projet : Méthodologie Agile avec Jira	52
8.1	Organisation Agile Scrum	52
8.1.1	Sprint 1 — Cadrage, Étalonnage & Ingestion	52
8.1.2	Sprint 2 — Qualité des Données & Streaming	53
8.1.3	Sprint 3 — Modélisation ML & MLflow	53
8.1.4	Sprint 4 — API, CI/CD & Déploiement	53
8.2	Backlog Produit et Traçabilité	54
9	Conclusion & Travaux Futurs	55
9.1	Résumé des Contributions	55
9.2	Limites	56
9.3	Travaux Futurs	56
9.4	Remarques Finales	57

List of Figures

2.1	Distribution des classes de trafic	6
2.2	Analyse des caractéristiques (Features) et exploration visuelle du dataset CICIDS2017.	7
2.3	Diagramme de l'architecture globale du système MLSecOps.	7
4.1	Corrélation entre les variables (1) pour l'entraînement du modèle XGBoost.	23
4.2	Corrélation entre les variables (2) et sélection des variables (Feature Selection).	24
4.3	Interface de suivi MLflow — liste des exécutions du modèle XGBoost. . . .	28
4.4	Page <i>Batch analyzer</i> du frontend, affichant le résultat de la prédiction par lots pour chaque enregistrement d'un fichier CSV téléversé : étiquette (<i>Benign/Attack</i>) et score de confiance associé, avec possibilité de filtrer et d'exporter les résultats.	30
5.1	Sortie de <code>docker compose ps</code> — les huit services de la plateforme à l'état Up.	33
5.2	Pipeline GitHub Actions — exécution réussie (pre-commit, flake8, pytest, build Docker).	34
5.3	Réponse JSON du point de terminaison <code>/health</code> , confirmant que le service est opérationnel (<code>"status": "ok"</code>) et que le modèle XGBoost a bien été chargé en mémoire (<code>"model_loaded": true</code>).	37
6.1	Graphe orienté acyclique (DAG) du pipeline Dagster.	38
7.1	Topologie du réseau Docker Compose (<code>mlops_network</code>).	44
7.2	Interface de requêtage (<i>Query</i>) de Prometheus, utilisée pour interroger de manière ad hoc les métriques exposées par le point de terminaison <code>/metrics</code> de FastAPI (par exemple <code>model_predictions_total</code> ou <code>anomalies_detected_total</code>).	46
7.3	Tableau de bord Grafana construit sur la source de données Prometheus, combinant une série temporelle des compteurs <code>anomalies_detected_total/anomalies_detected</code> , un histogramme de la charge de requêtes, et deux panneaux de type jauge (<i>stat</i>) affichant respectivement 0 anomalie récente et 82 prédictions cumulées.	46
7.4	Documentation interactive FastAPI (Swagger UI) — endpoints <code>/health</code> , <code>/predict</code> , <code>/predict/batch</code> et <code>/metrics</code>	47
7.5	Pages principales du frontend React de la plateforme MLSecOps.	50
7.6	Page de configuration (Settings) — URL de base de l'API.	50
7.7	Interface frontend de la plateforme MLSecOps (Section Hero).	51

8.1	Tableau Jira du Sprint 1 (<i>Cadrage, Étal & Ingest</i> , 5–11 août) : cadrage de la problématique, des objectifs et de la Data Strategy, mise en place du Product Backlog initial et premières tâches d’ingestion du jeu de données CICIDS2017.	52
8.2	Tableau Jira du Sprint 2 (<i>Data Quality & Ingest</i> , 12–18 août) : nettoyage et normalisation des flux, transformations dbt, et mise en place des portes de qualité des données.	53
8.3	Tableau Jira du Sprint 3 (<i>Modélisation ML & MLflow</i> , 19–25 août) : sélection des caractéristiques, prévention de la fuite de données et suivi des expériences avec MLflow.	53
8.4	Tableau Jira du Sprint 4 (<i>API, CI/CD & Deployment</i> , 26 août–1 ^{er} septembre) : exposition du modèle via FastAPI, sécurisation, conteneurisation et automatisation CI/CD.	53
8.5	Extrait du backlog produit Jira (plus de 70 éléments), incluant les tickets de fonctionnalités et de correction de bogues répartis sur l’ensemble des sprints du projet.	54

List of Tables

2.1	Services Docker Compose et leurs rôles dans la plateforme MLSecOps. . . .	6
2.2	Pages frontend et leurs fonctionnalités.	10
3.1	Catégories de trafic CICIDS2017 et leur prévalence.	13
3.2	Règles de qualité des données CSV brutes.	17
3.3	Règles de qualité des données des enregistrements nettoyés.	17
3.4	Règles de qualité des données de la sortie d'inférence.	18
4.1	Configuration des hyperparamètres XGBoost.	27
5.1	Crochets de pré-validation et leurs implications en matière de sécurité. . .	32
7.1	Volumes Docker et leurs objectifs.	43

Chapter 1

Introduction

1.1 Contexte et Motivation

La dernière décennie a été le témoin d’une accélération sans précédent de l’adoption de l’apprentissage automatique (ML) dans presque tous les secteurs industriels. Des moteurs de recommandation propulsant le commerce électronique aux modèles d’apprentissage profond d’aide au diagnostic médical, le ML est passé d’une curiosité académique à une infrastructure de production critique. Cependant, cette adoption rapide a mis en évidence une lacune critique : alors que le génie logiciel dispose de décennies de pratiques établies pour construire, tester, déployer et surveiller des applications — collectivement connues sous le nom de DevOps — l’écosystème ML est resté comparativement immature dans ces disciplines opérationnelles.

Le **MLOps** (Machine Learning Operations) a émergé en réponse à cette lacune, adaptant les principes DevOps aux défis uniques des systèmes ML. Contrairement aux logiciels traditionnels, les systèmes ML font face à un axe de complexité supplémentaire : les données. Les modèles se dégradent silencieusement lorsque les distributions d’entrée évoluent (data drift), les écarts entre l’entraînement et la production introduisent des bogues subtils, et la reproductibilité des expériences nécessite le contrôle simultané des versions du code, des données, des hyperparamètres et des environnements.

Le **DataOps**, une discipline parallèle, s’attaque aux défis opérationnels des pipelines de données eux-mêmes : s’assurer que les données sont ingérées de manière fiable, transformées de manière reproductible, validées selon des contrats de qualité et livrées aux consommateurs en aval en temps opportun et de manière fiable. La convergence du DataOps et du MLOps reconnaît qu’un modèle n’est bon que dans la mesure où les données qu’il consomme le sont.

Pourtant, même le cadre combiné MLOps + DataOps s’avère insuffisant lorsque le système déployé fonctionne dans un environnement hostile. La détection d’intrusions réseau, la prévention de la fraude et la recherche de menaces en cybersécurité sont des domaines où :

- Le modèle lui-même peut être la cible d’attaques adverses (évasion, empoisonnement, extraction de modèle).
- Le pipeline de données peut être compromis (injection de données, manipulation de schéma).
- L’infrastructure de service peut être exploitée (évasion de conteneur, abus d’API).

- Le code source doit résister aux attaques de la chaîne d’approvisionnement (vulnérabilités des dépendances, hooks pre-commit malveillants).

Le **MLSecOps** — l’intégration de pratiques de sécurité dans le cycle de vie du MLOps — répond à ces préoccupations en intégrant directement dans le pipeline des portes de sécurité, des analyses automatisées, la validation des entrées et les principes du moindre privilège. C’est la philosophie qui anime la plateforme présentée dans ce rapport.

1.2 Problématique

La détection d’intrusions réseau est un problème classique de cybersécurité dans lequel un système automatisé doit distinguer en temps réel le trafic réseau bénin du trafic malveillant. Le jeu de données CICIDS2017, créé par l’Institut Canadien pour la Cybersécurité, fournit des enregistrements de flux réseau étiquetés générés en simulant des scénarios d’attaques réalistes (DDoS, balayage de ports, attaques par force brute, injection SQL, botnets, infiltration, etc.) contre une topologie de réseau réaliste.

Le défi ne consiste pas simplement à entraîner un classificateur précis — il s’agit de construire un *système complet* qui :

1. Ingère des données de flux réseau brutes en temps réel depuis plusieurs sources.
2. Valide la qualité des données à chaque étape du pipeline pour prévenir le "garbage-in-garbage-out".
3. Transforme et prépare les données pour la consommation du modèle sans divulguer d’identifiants sensibles.
4. Entraîne, évalue et versionne les modèles ML avec un suivi complet des expériences.
5. Sert les prédictions via une API REST à faible latence avec une surveillance de l’état de santé.
6. Surveille le comportement du modèle en production grâce à des métriques et des tableaux de bord.
7. Orchestre l’ensemble du pipeline en tant que DAG reproductible et planifiable.
8. Applique la sécurité aux niveaux du code, du conteneur et de l’infrastructure.
9. Fournit une interface conviviale pour permettre aux analystes de sécurité d’interagir avec le système.

1.3 Objectifs

Les objectifs spécifiques de ce projet sont les suivants :

1. **Concevoir et implémenter un pipeline d’ingestion de données en streaming** en utilisant Kafka/Redpanda qui simule le trafic réseau en temps réel à partir des fichiers CSV du CICIDS2017, avec un débit et une taille de lot configurables.
2. **Construire un framework de qualité de données multi-étapes** (831 lignes de code Python validé) qui applique des règles d’exhaustivité, de validité et d’intégrité à trois étapes du pipeline : ingestion brute, enregistrements nettoyés et sortie d’inférence — en arrêtant le pipeline en cas d’échec.
3. **Entraîner un détecteur d’anomalies réseau XGBoost** avec une validation croisée à k-plis stratifiée, une pondération des échantillons atténuée par racine carrée

pour les classes d'attaques rares, et une prévention stricte de la fuite de données par la suppression explicite des colonnes sensibles.

4. **Suivre toutes les expériences ML** avec MLflow, en enregistrant les hyperparamètres, les métriques et les artefacts du modèle vers un serveur de suivi centralisé avec un backend SQLite.
5. **Servir les prédictions via une API REST FastAPI** avec validation des schémas Pydantic (33 caractéristiques de flux réseau), des endpoints uniques et par lots, des sondes de santé, et une instrumentation de métriques Prometheus.
6. **Orchestrer l'ensemble du pipeline avec Dagster**, en définissant les actifs de données (assets) avec des graphes de dépendances explicites, des capteurs de système de fichiers pour un déclenchement automatique, des planifications cron horaires, et des portes de qualité qui peuvent arrêter le pipeline.
7. **Conteneuriser tous les services avec Docker Compose**, y compris Dagster (serveur web + démon), dbt, Redpanda, MLflow, FastAPI, Prometheus, et Grafana — tous communiquant via un réseau bridge isolé.
8. **Implémenter des contrôles de sécurité** incluant des hooks pre-commit (Black, espaces en fin de ligne, validation YAML, détection de gros fichiers), un pipeline CI/CD GitHub Actions (linting flake8, pytest, construction d'images Docker), des utilisateurs de conteneurs non-root, et un accès API restreint par CORS.
9. **Développer un frontend en React + TypeScript** (Vite + TailwindCSS) comportant six pages : Tableau de bord (Dashboard), Inspecteur de flux, Analyse par lots, Alertes, État du système, et Paramètres.
10. **Implémenter la transformation des données avec dbt** sur DuckDB, fournissant des modèles SQL testés et versionnés pour la couche de staging.

1.4 Structure du Rapport

Le reste de ce rapport est organisé comme suit :

Chapitre 2

présente l'architecture globale du système et la pile technologique, en expliquant le rôle et la justification de chaque composant.

Chapitre 3

propose une analyse approfondie de la couche DataOps : ingestion des données, streaming, nettoyage, validation de la qualité, transformations dbt, et préparation des données.

Chapitre 4

couvre l'entraînement du modèle, l'ingénierie des caractéristiques, le suivi des expériences avec MLflow, et le module d'inférence des prédictions.

Chapitre 5

analyse l'intégration de la sécurité à travers la plateforme : hooks pre-commit, CI/CD, stratégie de test, sécurité des conteneurs, et prévention des fuites de données.

Chapitre 6

détaille la couche d'orchestration Dagster : définition des actifs (assets), capteurs, planifications, et portes de qualité.

Chapitre 7

décrit l'architecture de déploiement (Docker Compose), la couche de service FastAPI, le frontend React, et la pile de surveillance Prometheus/Grafana.

Chapitre 8

présente la gestion de projet en méthodologie Agile Scrum avec Jira : organisation en sprints, backlog produit et traçabilité des tickets.

Chapitre 9

conclut le rapport par un résumé des contributions et des pistes pour les travaux futurs.

Chapter 2

Architecture du Système & Technologies

2.1 Aperçu de l'Architecture Globale

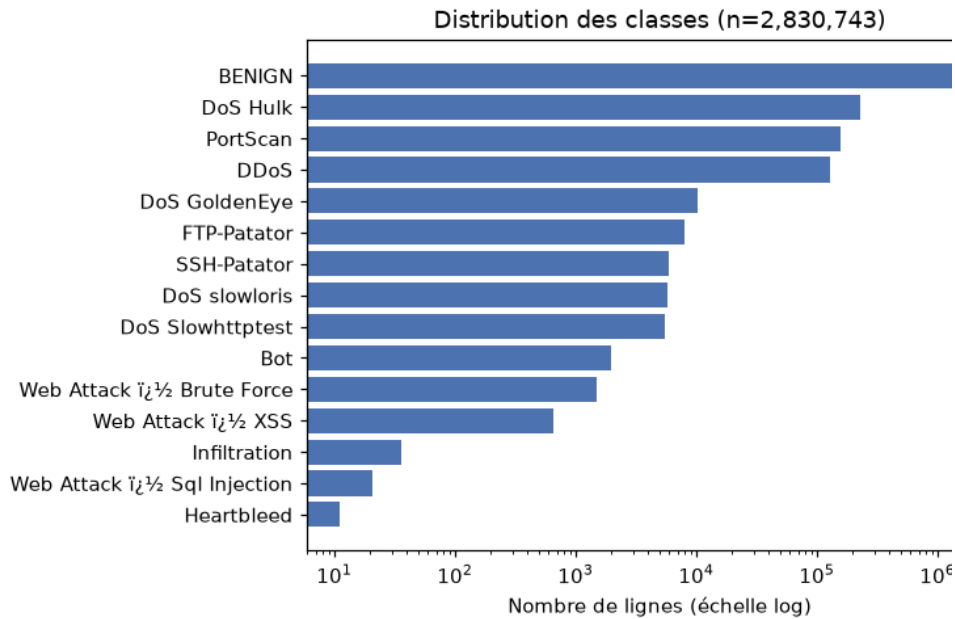
La Plateforme MLSecOps suit une **architecture en microservices** dans laquelle chaque préoccupation fonctionnelle — ingestion des données, traitement des flux, entraînement ML, service de modèle, orchestration, surveillance, et interface utilisateur — est encapsulée dans son propre service conteneurisé. Ces services communiquent via un réseau Docker bridge isolé (`mlops_network`) et sont définis de manière déclarative dans un seul manifeste `docker-compose.yml`.

La Figure 2.3 présente une vue de haut niveau de l'architecture du système, illustrant le flux de données depuis l'ingestion brute de CSV, à travers le streaming, le nettoyage, la validation de la qualité, l'entraînement du modèle, l'inférence en temps réel, le service API, jusqu'à la visualisation sur le tableau de bord.

L'architecture est composée de **huit services Docker** orchestrés par Docker Compose :

Table 2.1: Services Docker Compose et leurs rôles dans la plateforme MLSecOps.

Service	Conteneur	Port	Rôle
dagster	dagster_orchestrator	4300	Serveur web Dagster (interface du pipeline)
dagster-daemon	dagster_daemon	—	Planificateur en arrière-plan & démon de capteurs
dbt	dbt_runner	—	Exécuteur de transformation dbt (DuckDB)
redpanda	redpanda	4305–4308	Broker de streaming compatible Kafka
mlflow	mlflow_server	4301	Suivi des expériences & registre de modèles
fastapi-app	fastapi_anomaly	4302	API REST pour l'inférence en temps réel
prometheus	prometheus	4303	Collecte de métriques & alertes
grafana	grafana	4304	Tableaux de bord des métriques

**Figure 2.1:** Distribution des classes de trafic

Comme illustré dans la figure ci-dessus, la distribution des classes de trafic au sein du jeu de données CICIDS2017 est fortement déséquilibrée. La majorité des flux réseau représente un trafic bénin, tandis que les vecteurs d'attaque, tels que DoS, DDoS et PortScan, constituent une fraction beaucoup plus faible. Ce déséquilibre significatif des classes nécessite un traitement spécialisé lors de la phase d'entraînement du modèle, tel que la pondération des échantillons atténuée par racine carrée discutée plus loin dans ce rapport, pour éviter que le classificateur ne soit biaisé en faveur de la classe majoritaire.

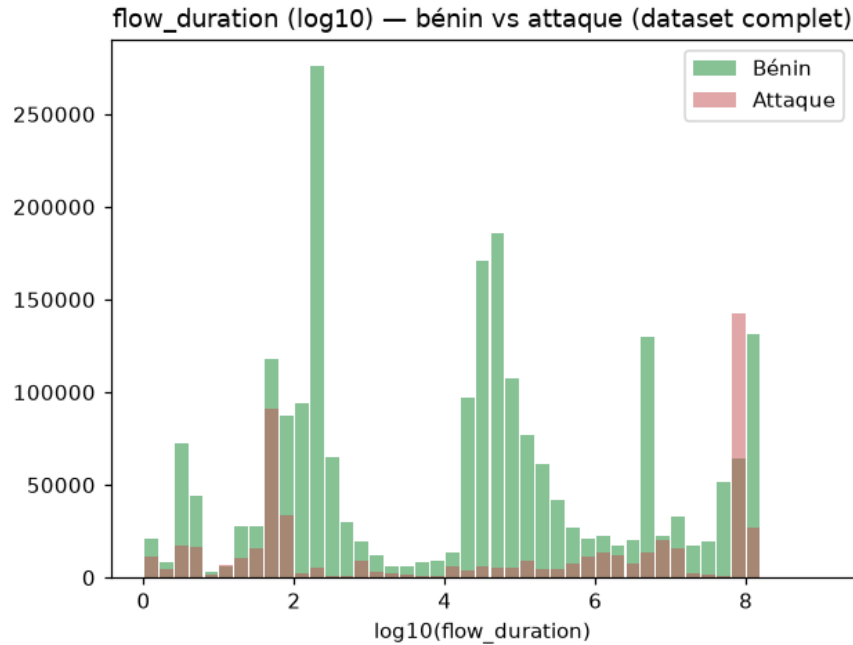


Figure 2.2: Analyse des caractéristiques (Features) et exploration visuelle du dataset CICIDS2017.

Suite à la distribution des classes, l'analyse des caractéristiques illustrée ci-dessus offre des perspectives plus approfondies sur les attributs sous-jacents des flux réseau. Les métriques visualisées mettent en évidence les variations à travers différents types de trafic, montrant comment certaines caractéristiques, telles que la longueur des paquets et les temps d'inter-arrivée, présentent des schémas distincts lors des scénarios d'attaque par rapport à un comportement réseau normal. Cette analyse exploratoire des données est cruciale pour guider le processus de sélection des caractéristiques et s'assurer que le modèle capture les indicateurs d'activité malveillante les plus pertinents.

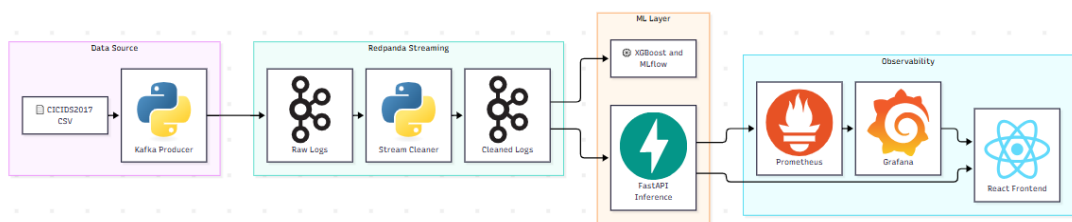


Figure 2.3: Diagramme de l'architecture globale du système MLSecOps.

Ce diagramme illustre le flux de données de bout en bout au sein de la plateforme MLSecOps : depuis l'ingestion des fichiers CSV du jeu de données CICIDS2017, en passant par la publication et la consommation via Redpanda (topics `raw-logs` et `cleaned-logs`), le nettoyage des flux, l'entraînement du modèle XGBoost avec suivi MLflow, jusqu'au service d'inférence FastAPI exposé à la couche de surveillance (Prometheus, Grafana) et à l'interface utilisateur React. Il offre une vue synthétique de l'interaction entre les composants DataOps, MLOps et de service, complétant la description textuelle de cette section.

2.2 Analyse Approfondie de la Pile Technologique

2.2.1 Python 3.10 — Environnement d’Exécution Principal

Python sert de langage principal pour le traitement des données, l’entraînement ML, le service d’API et l’orchestration. Le projet cible explicitement Python 3.10 (comme spécifié dans les Dockerfiles et la configuration CI) pour tirer parti des indications de type modernes (type hints), du pattern matching, et des améliorations de performances. Les bibliothèques Python clés incluent :

- **pandas** ≥ 2.0 et **numpy** ≥ 1.24 pour la manipulation de données tabulaires et le calcul numérique.
- **scikit-learn** ≥ 1.3 pour les utilitaires de prétraitement (**StratifiedKFold**, **train_test_split**, **compute_sample_weight**) et les métriques d’évaluation.
- **xgboost** ≥ 2.0 pour la classification par arbres boostés par gradient.
- **joblib** ≥ 1.3 pour la sérialisation efficace des modèles.
- **kafka-python** ≥ 2.0 pour l’intégration des producteurs/consommateurs Kafka.
- **gdown** pour le téléchargement automatisé du jeu de données depuis Google Drive.

2.2.2 FastAPI — Framework d’API REST

FastAPI a été choisi pour l’API d’inférence en raison de sa combinaison de hautes performances (basée sur Starlette et Uvicorn ASGI), sa documentation OpenAPI automatique, sa validation native avec Pydantic, et son support asynchrone. Le fichier `app/main.py` du projet définit une application FastAPI avec :

- Un hook de cycle de vie (lifespan) basé sur **asynccontextmanager** qui charge le modèle XGBoost une seule fois au démarrage et le libère à l’arrêt — éliminant ainsi le coût de chargement du modèle pour chaque requête.
- Quatre endpoints : `/health` (sonde de disponibilité), `/predict` (prédiction de flux unique), `/predict/batch` (prédiction par lots), et `/metrics` (endpoint de scraping Prometheus).
- Un middleware CORS configuré pour les origines du frontend (`localhost:5173` et `localhost:3000`).

2.2.3 Dagster — Orchestration du Pipeline

Dagster a été sélectionné plutôt que d’autres alternatives (Airflow, Prefect) pour son paradigme *centré sur les actifs* (asset-centric), qui traite les sorties de pipeline comme des entités de premier ordre plutôt que de se concentrer sur les tâches. Cette conception s’aligne naturellement avec les workflows ML où l’artefact (jeu de données nettoyé, modèle entraîné) est plus important que la tâche qui l’a produit. La plateforme utilise :

- **Des actifs définis par logiciel** (`@asset`) pour modéliser les étapes d’ingestion en streaming, de nettoyage, et d’inférence.
- **Des dépendances d’actifs** (`deps=[...]`) pour définir l’ordre d’exécution du DAG.
- **Des capteurs de système de fichiers** pour déclencher automatiquement des exécutions du pipeline lorsque de nouveaux fichiers CSV apparaissent dans `data/raw/`.
- **Des planifications cron horaires** comme filet de sécurité pour garantir un traitement périodique.

- **Des annotations de métadonnées** (`MetadataValue`) pour afficher les métriques du pipeline directement dans l'interface de Dagster.

2.2.4 Redpanda — Backbone de Streaming

Redpanda sert de broker de streaming compatible Kafka, offrant plusieurs avantages par rapport à Apache Kafka pour le développement et les tests : il ne nécessite pas de JVM, ni de ZooKeeper, fournit un déploiement sous forme d'un binaire unique, et offre une consommation de ressources nettement inférieure. La plateforme configure Redpanda avec :

- Un mode à nœud unique (`-smp 1 -memory 1G`) pour le développement.
- Une configuration à double écouteur : `PLAINTEXT` pour la communication intra-conteneur et `OUTSIDE` pour l'accès hôte.
- Trois topics Kafka : `raw-logs` (données ingérées), `cleaned-logs` (données traitées), et `alerts` (résultats d'inférence).

2.2.5 dbt + DuckDB — Transformation des Données

dbt (Data Build Tool) fournit des transformations SQL testables et versionnées. Le projet utilise `dbt-duckdb` comme adaptateur, tirant parti du moteur d'analyse embarqué de DuckDB pour éviter le surcoût d'un serveur de base de données distinct. Le projet dbt (`dbt_project/`) inclut :

- Un modèle de staging (`stg_network_logs`) qui normalise les données brutes des journaux réseau.
- Des tests de niveau schéma (`unique`, `not_null`, `accepted_values`) définis dans `schema.yml`.
- Des profils configurés pour écrire dans `/app/dbt_project/target/duck.db`.

2.2.6 MLflow — Suivi des Expériences & Registre de Modèles

MLflow fournit l'infrastructure de suivi des expériences, fonctionnant comme un conteneur dédié avec :

- Un magasin backend SQLite (`sqlite:///app/mlflow.db`) pour les métadonnées des expériences.
- Une racine d'artefacts locale (`/app/mlruns`) sauvegardée par un volume Docker pour la persistance du modèle.
- Une API HTTP accessible à tous les conteneurs via `http://mlflow:5000`.

Le pipeline d'entraînement enregistre les hyperparamètres, les métriques de validation croisée, les métriques de test et le modèle XGBoost entraîné sous forme d'artefacts dans des exécutions nommées.

2.2.7 Prometheus + Grafana — Surveillance

Prometheus scrape (récupère) les métriques de l'application FastAPI toutes les 15 secondes, comme configuré dans `prometheus.yml` :

```
1 global:
```

```

2  scrape_interval: 15s
3
4  scrape_configs:
5    - job_name: 'fastapi_anomaly_detection'
6      metrics_path: '/metrics'
7      static_configs:
8        - targets: ['fastapi-app:8000']

```

Listing 2.1: Configuration de scraping Prometheus

Trois métriques personnalisées sont instrumentées dans l'application FastAPI :

- `model_predictions_total` (Compteur) — total des prédictions, étiquetées par version du modèle.
- `anomalies_detected_total` (Compteur) — total des anomalies détectées.
- `model_inference_seconds` (Histogramme) — distribution de la latence d'inférence.

Grafana fournit des tableaux de bord par-dessus Prometheus, accessibles au port 4304 avec les identifiants par défaut.

2.2.8 React + TypeScript + Vite — Frontend

L'interface utilisateur est construite avec React 19, TypeScript 6 et Vite 8, en utilisant TailwindCSS pour le style et Recharts pour la visualisation des données. Le frontend est structuré comme une application à page unique (SPA) avec six pages :

Table 2.2: Pages frontend et leurs fonctionnalités.

Route	Page	Description
/	DashboardPage	Vue d'ensemble avec statistiques de prédiction et graphiques
/inspect	InspectorPage	Saisie manuelle des 33 caractéristiques de flux pour une prédiction unique
/batch	BatchPage	Téléchargement CSV pour prédiction par lots avec suivi de progression
/alerts	AlertsPage	Alertes en temps réel pour les anomalies détectées
/status	StatusPage	Santé du système : statut de l'API, état de chargement du modèle
/settings	SettingsPage	Configuration de l'URL de base de l'API

2.2.9 Docker — Stratégie de Conteneurisation

Le projet utilise deux Dockerfiles avec des objectifs distincts :

Dockerfile Racine (Dockerfile) — Utilisé par le service FastAPI. Il suit les meilleures pratiques de sécurité :

```

1 FROM python:3.10-slim
2
3 ENV PYTHONDONTWRITEBYTECODE=1 \

```

```
4     PYTHONUNBUFFERED=1
5
6 WORKDIR /app
7
8 COPY requirements.txt .
9 RUN pip install --no-cache-dir --upgrade pip && \
10     pip install --no-cache-dir -r requirements.txt
11
12 COPY . .
13
14 # Run as a non-root user for security best practices
15 RUN useradd -m appuser && chown -R appuser:appuser /app
16 USER appuser
17
18 CMD ["python", "src/ingestion/ingest.py"]
```

Listing 2.2: Dockerfile racine pour le service FastAPI

Les décisions clés en matière de sécurité incluent :

- **Utilisateur non-root** : Le conteneur s'exécute en tant qu'utilisateur `appuser`, réduisant ainsi le rayon d'impact de toute vulnérabilité d'évasion de conteneur.
- **Image de base slim** : `python:3.10-slim` minimise la surface d'attaque en excluant les paquets système inutiles.
- **Sans cache** : `-no-cache-dir` empêche pip de stocker les paquets téléchargés, réduisant la taille de l'image et éliminant le cache obsolète comme vecteur d'attaque.
- **Sortie non tamponnée** : Garantit que les journaux (logs) sont immédiatement disponibles pour le débogage et la surveillance.

Dockerfile Source (`src/Dockerfile`) — Utilisé par les services Dagster, dbt et MLflow. Il inclut des dépendances système supplémentaires (`build-essential`, `git`, `curl`) requises pour la compilation des packages Python et des outils d'orchestration.

Chapter 3

DataOps & Ingénierie du Pipeline

Ce chapitre fournit une analyse complète de la couche de données de la plateforme MLSecOps, couvrant le parcours complet des données depuis les fichiers CSV bruts, en passant par l'ingestion en streaming, le nettoyage en temps réel, la validation de la qualité multi-étapes, et la transformation basée sur dbt.

3.1 Source de Données : Dataset CICIDS2017

Le jeu de données CICIDS2017 (Canadian Institute for Cybersecurity Intrusion Detection System 2017) est l'un des jeux de données de référence les plus largement utilisés dans la recherche sur la détection d'intrusions réseau. Il contient plus de 2,8 millions d'enregistrements de flux réseau étiquetés générés sur cinq jours d'activité réseau simulée, avec à la fois du trafic bénin et des scénarios d'attaques réalistes.

Le jeu de données comprend 79 caractéristiques (features) par enregistrement de flux, englobant :

- **Statistiques de paquets** : Comptages, longueurs, moyennes et écarts types pour les paquets entrants et sortants (forward and backward).
- **Temps d'inter-arrivée (IAT)** : Temps moyen, écart type, maximum et minimum entre les paquets.
- **Drapeaux TCP** : Comptages des drapeaux FIN, SYN, RST, PSH, ACK, URG, CWE et ECE.
- **Métriques au niveau du flux** : Durée, octets/seconde, paquets/seconde, statistiques sur la taille des paquets.
- **Métriques de sous-flux et transfert de masse** : Comptages de paquets/octetes de sous-flux et taux de transfert de masse.
- **Métadonnées de connexion** : Port de destination, tailles de fenêtres initiales, temps actif/inactif.
- **Étiquette (Label)** : Classification du trafic (BENIGN ou l'une des 14 catégories d'attaques).

Les 15 catégories de trafic dans le jeu de données sont :

Table 3.1: Catégories de trafic CICIDS2017 et leur prévalence.

Catégorie	Description
BENIGN	Trafic réseau normal (classe majoritaire)
DoS Hulk	Déni de service via l'outil Hulk
PortScan	Balayage de ports réseau
DDoS	Déni de service distribué
DoS GoldenEye	DoS via attaque HTTP GoldenEye
FTP-Patator	Connexion FTP par force brute
SSH-Patator	Connexion SSH par force brute
DoS slowloris	Attaque DoS HTTP lente
DoS Slowhttptest	DoS HTTP lent sur corps/en-tête
Bot	Activité C&C de Botnet
Web Attack – Brute Force	Attaque HTTP par force brute
Web Attack – XSS	Attaque par script inter-site (XSS)
Web Attack – SQL Injection	Attaque par injection SQL
Infiltration	Infiltration réseau
Heartbleed	Exploit Heartbleed OpenSSL

3.2 Pipeline d'Ingestion des Données

3.2.1 Téléchargement Automatisé du Dataset

Le module `src/ingestion/download_data.py` automatise l'acquisition du jeu de données depuis Google Drive en utilisant la bibliothèque `gdown` :

```

1 def main():
2     folder_id = "1wYlA6xd1dWkzhCBanogywvn85_Tg5cuY"
3     output_dir = os.path.abspath(
4         os.path.join(os.path.dirname(__file__), "../..data/raw/")
5     )
6     os.makedirs(output_dir, exist_ok=True)
7     gdown.download_folder(
8         id=folder_id, output=output_dir,
9         quiet=False, use_cookies=False
10    )

```

Listing 3.1: Téléchargement automatisé du dataset depuis Google Drive

Cette approche garantit la reproductibilité : tout membre de l'équipe peut remplir le répertoire `data/raw/` avec une seule commande, sans gestion manuelle des fichiers.

3.2.2 Ingestion en Streaming avec Kafka Producer

Le module `src/streaming/producer.py` simule le trafic réseau en temps réel en lisant des fichiers CSV locaux en blocs configurables et en publiant chaque enregistrement sous forme de message JSON dans le topic Kafka `raw-logs`.

```

1 def get_kafka_producer():
2     for attempt in range(5):

```

```

3         try:
4             return KafkaProducer(
5                 bootstrap_servers=os.environ.get(
6                     "KAFKA_BOOTSTRAP_SERVERS", "localhost:9092"
7                 ),
8                 value_serializer=lambda v: json.dumps(v).encode("utf-8"),
9                 acks=1,
10                linger_ms=50,
11                batch_size=262144,
12                compression_type="gzip",
13            )
14        except KafkaError as e:
15            logger.warning(
16                f"Failed to connect to Kafka, attempt {attempt + 1}/5: {e}"
17            )
18            time.sleep(2**attempt)
19        raise RuntimeError("Could not connect to Kafka after multiple
    retries.")

```

Listing 3.2: Configuration du producteur Kafka avec logique de réessai et compression

Plusieurs décisions de conception méritent des explications :

- **Réessai avec backoff exponentiel** (lignes 8–10) : Le producteur tente de se connecter jusqu'à 5 fois avec des délais de 2^n secondes (1, 2, 4, 8, 16 secondes), offrant une résilience contre les problèmes d'ordre de démarrage des conteneurs dans Docker Compose.
- **Compression Gzip** (`compression_type="gzip"`) : Réduit la consommation de bande passante réseau de 60 à 80 % pour les payloads JSON, ce qui est critique lors de la publication de millions d'enregistrements.
- **Configuration par lots (Batch)** (`batch_size=262144`, `linger_ms=50`) : Le producteur regroupe jusqu'à 256 Ko de messages et attend jusqu'à 50 ms avant l'envoi, améliorant considérablement le débit par rapport à des envois par message individuel.
- **Niveau d'accusé de réception (Acknowledgment)** (`acks=1`) : Le broker leader accuse réception de chaque lot, offrant un équilibre entre durabilité (vs. `acks=0`) et latence (vs. `acks=all`).

L'étape de nettoyage (sanitisation) des données avant la publication est critique :

```

1 def _sanitize_chunk(chunk):
2     chunk = chunk.replace([np.inf, -np.inf], np.nan)
3     chunk = chunk.where(pd.notnull(chunk), None)
4     return chunk

```

Listing 3.3: Nettoyage des données avant la publication Kafka

Le jeu de données CICIDS2017 contient des valeurs infinies dans les colonnes de débit de flux (ex. Flow Bytes/s) chaque fois que la durée du flux est nulle (division par zéro). Ces valeurs ne peuvent pas être sérialisées en JSON ; elles sont donc remplacées par NaN puis par None pour assurer la compatibilité JSON.

3.3 Pipeline de Nettoyage de Flux (Stream Cleaning)

Le module `src/streaming/cleaner.py` consomme les messages du topic `raw-logs`, applique un nettoyage au niveau de l'enregistrement et publie les enregistrements normalisés dans le topic `cleaned-logs` tout en les écrivant simultanément dans un fichier d'export JSONL local.

3.3.1 Logique de Nettoyage des Enregistrements

La fonction `clean_record()` effectue trois transformations critiques :

```

1  def clean_record(record):
2      cleaned = {}
3      for k, v in record.items():
4          new_k = (
5              k.strip()
6              .lower()
7              .replace(" ", "_")
8              .replace("/", "_")
9              .replace("-", "_")
10             .replace(".", "_")
11         )
12         cleaned[new_k] = _coerce_value(v)
13
14     label = cleaned.get("label", "benign")
15     label = _normalize_label(label)
16     cleaned["label"] = label
17     cleaned["is_attack"] = 1 if label != "benign" else 0
18     cleaned["ingested_at"] = time.time()
19     return cleaned

```

Listing 3.4: Nettoyage d'enregistrement : normalisation, coercition et traitement de l'étiquette

1. **Normalisation des clés** (lignes 3–11) : Tous les noms de colonnes sont convertis en snake_case minuscule. Le CSV original du CICIDS2017 utilise des noms à casse mixte avec des espaces (ex. " Flow Duration", " Fwd Packet Length Max"). Cette normalisation est essentielle pour un accès cohérent en aval.
2. **Coercition des valeurs** (`_coerce_value`) : Gère None, NaN, Infinity et leurs représentations sous forme de chaînes de caractères, en les convertissant en 0.0. Les valeurs de chaîne pouvant être analysées comme des flottants sont converties ; les autres sont préservées.
3. **Normalisation de l'étiquette** : La fonction `_normalize_label()` gère les artefacts d'encodage des chaînes UTF-8 décodées depuis cp1252 (caractères tiret `xe2`, `u20ac`, `u201c`) et normalise toutes les étiquettes en ASCII minuscule.
4. **Indicateur d'attaque binaire** : Un champ dérivé `is_attack` (0 ou 1) est ajouté pour un usage direct par le pipeline ML.
5. **Horodatage d'ingestion** : `ingested_at` enregistre le temps de traitement pour le traçage des données (data lineage).

3.3.2 Sondage Inactif et Arrêt Doux (Graceful Shutdown)

Le nettoyeur (cleaner) implémente un mécanisme sophistiqué de détection d'inactivité pour déterminer quand arrêter la consommation :

- `KAFKA_IDLE_POLLS_LIMIT` (défaut 5) : Nombre maximum de sondages (polls) vides consécutifs avant l'arrêt.
- `KAFKA_IDLE_TIMEOUT_S` (défaut 120s dans Docker) : Durée de veille (sleep) entre les sondages vides.
- `KAFKA_POLL_TIMEOUT_MS` (défaut 5000ms) : Délai de sondage Kafka par lot.
- `EXPORT_APPEND` (défaut false) : Contrôle si le fichier d'export JSONL est annexé ou écrasé.

Le modèle de remplacement de fichier atomique (`os.replace(target_path, export_file)`) garantit que les consommateurs en aval ne lisent jamais un fichier d'export partiellement écrit.

3.4 Framework de Qualité des Données

Le module `src/quality/data_quality.py` (831 lignes) implémente un framework complet de validation de la qualité des données en trois étapes. C'est l'une des caractéristiques les plus distinctives de la plateforme, incarnant le principe selon lequel *la qualité des données est une préoccupation de premier ordre en MLSecOps*.

3.4.1 Architecture

Le framework est construit autour de trois abstractions principales :

- **RuleResult** : Une classe de données (dataclass) représentant le résultat d'un contrôle de règle de qualité unique, incluant le nom de la règle, la catégorie (exhaustivité, validité, intégrité), le statut de réussite/échec, un message lisible par l'homme, et un dictionnaire de détails.
- **QualityReport** : Un rapport agrégé pour une étape de pipeline qui accumule les résultats des règles, suit le nombre de réussites/échecs et fournit à la fois des résumés textuels et une sérialisation JSON.
- **DataQualityError** : Une exception personnalisée contenant le `QualityReport` défaillant, permettant aux actifs Dagster d'arrêter le pipeline avec des informations de diagnostic complètes.

3.4.2 Étape 1 : Validation du CSV Brut

La fonction `validate_raw_csv()` valide les DataFrames CICIDS2017 par rapport à douze règles réparties sur trois catégories :

Table 3.2: Règles de qualité des données CSV brutes.

ID	Catégorie	Règle	Description
C1	Exhaustivité	row_count_minimum	Le dataset doit contenir ≥ 1 ligne de données
C2	Exhaustivité	column_count_match	Exactement 79 colonnes requises
C3	Exhaustivité	critical_columns_present	Le Port de destination et l'Étiquette (Label) doivent exister
C4	Exhaustivité	label_null_rate	Taux de valeurs nulles pour le label $\leq 5\%$
C5	Exhaustivité	no_fully_empty_rows	Aucune ligne avec uniquement des NaN
C6	Exhaustivité	destination_port_null	Taux de valeurs nulles pour le port $\leq 5\%$
V1	Validité	port_range_valid	Ports compris dans $[0, 65535]$
V2	Validité	label_values_valid	Uniquement les étiquettes CICIDS2017 connues
V3	Validité	no_infinite_values	Aucun $\pm\infty$ dans les colonnes numériques
V4	Validité	non_negative_numerics	Colonnes paquets/flux ≥ 0
I1	Intégrité	duplicate_row_rate	Ratio de doublons $\leq 5\%$
I2	Intégrité	schema_column_match	Les colonnes correspondent au schéma CICIDS2017

3.4.3 Étape 2 : Validation des Enregistrements Nettoyés

La fonction `validate_cleaned_records()` valide les enregistrements JSONL produits par le nettoyeur de flux (stream cleaner) :

Table 3.3: Règles de qualité des données des enregistrements nettoyés.

ID	Catégorie	Règle	Description
C1	Exhaustivité	record_count_minimum	≥ 1 enregistrement nettoyé
C2	Exhaustivité	critical_keys_present	Clés : destination_port, label, flow_duration, total_fwd_packets
V1	Validité	keys_normalized	Toutes les clés correspondent à l'expression régulière <code>[a-z0-9][a-z0-9_/*]</code>
V2	Validité	port_values_valid	Ports compris dans $[0, 65535]$
V3	Validité	cleaned_label_values	Étiquettes (Labels) dans l'ensemble VALID_LABELS
I1	Intégrité	record_field_count	≥ 10 champs par enregistrement

3.4.4 Étape 3 : Validation de la Sortie d'Inférence

La fonction `validate_inference_output()` valide les enregistrements d'anomalies produits par le moteur d'inférence ML :

Table 3.4: Règles de qualité des données de la sortie d'inférence.

ID	Catégorie	Règle	Description
C1	Exhaustivité	ml_metadata_present	ml_confidence_score, ml_model_version présents
V1	Validité	confidence_range_valid	Score $\in [0.0, 1.0]$
V2	Validité	model_version_valid	Chaîne de version non vide
V3	Validité	threshold_consistency	Toutes les anomalies dépassent le seuil de 0.80
I1	Intégrité	anomaly_count_plausible	Nombre d'anomalies $\leq$ total traité

3.5 Couche de Transformation dbt

Le répertoire `dbt_project/` contient un projet dbt nommé `cicids2017_transformations` qui fournit des transformations SQL testables et versionnées sur un entrepôt (warehouse) DuckDB.

3.5.1 Configuration du Projet

```

1 name: cicids2017_transformations
2 version: '1.0'
3 profile: mlsecops
4 config-version: 2
5 model-paths: ["models"]
6 target-path: "target"
7 clean-targets: ["target", "dbt_modules"]

```

Listing 3.5: Configuration du projet dbt (`dbt_project.yml`)

```

1 mlsecops:
2   target: dev
3   outputs:
4     dev:
5       type: duckdb
6       path: /app/dbt_project/target/duck.db
7       threads: 1
8       keep_temp_files: false

```

Listing 3.6: Configuration du profil dbt avec adaptateur DuckDB

Le choix de DuckDB comme moteur d'entrepôt est délibéré : c'est une base de données analytique embarquée qui ne nécessite aucun processus serveur, ce qui la rend idéale pour les environnements conteneurisés où l'efficacité des ressources est primordiale. DuckDB offre un stockage orienté colonnes, une exécution vectorisée et un support SQL complet tout en s'exécutant entièrement au sein du processus dbt.

3.5.2 Modèle de Staging

Le modèle de staging (mise en attente) `stg_network_logs.sql` définit la couche initiale de transformation :

```
1 select
2   1 as id,
3   'demo_event' as event_name,
4   '2026-01-01 00:00:00'::timestamp as event_time,
5   '192.168.0.1' as source_ip,
6   '10.0.0.1' as destination_ip,
7   80 as destination_port,
8   0 as is_attack
```

Listing 3.7: Modèle de staging dbt pour les journaux réseau

3.5.3 Tests au Niveau du Schéma

Le fichier `schema.yml` définit des tests complets pour le modèle de staging :

```
1 version: 2
2 models:
3   - name: stg_network_logs
4     description: >
5       Modèle de staging pour les journaux de trafic réseau CICIDS2017.
6       Fournit des colonnes nettoyées et typées à partir des données
7         ingérées brutes.
8     columns:
9       - name: id
10         tests: [unique, not_null]
11       - name: event_name
12         tests: [not_null]
13       - name: event_time
14         tests: [not_null]
15       - name: source_ip
16         tests: [not_null]
17       - name: destination_ip
18         tests: [not_null]
19       - name: destination_port
20         tests: [not_null]
21       - name: is_attack
22         tests:
23           - not_null
24           - accepted_values:
25               values: [0, 1]
```

Listing 3.8: Tests de schéma dbt avec unicité, non-null, et valeurs acceptées

Ces tests garantissent que chaque colonne n'est pas nulle, que la colonne `id` est unique et que la colonne `is_attack` ne contient que des valeurs binaires valides. Lorsque `dbt test` est exécuté, toute violation déclenche une erreur, empêchant la propagation de données invalides en aval.

3.6 Préparation des Données pour le ML

Deux modules spécialisés dans `src/prepare/` comblent le fossé entre le pipeline de streaming/nettoyage et le pipeline d'entraînement ML :

3.6.1 Conversion de JSONL en Parquet

Le module **prepare_data.py** lit l'export JSONL nettoyé, applique l'extraction de caractéristiques (en utilisant la même fonction `extract_features()` que celle utilisée lors de l'entraînement), et écrit le résultat sous la forme d'un fichier Parquet :

```

1 def process_jsonl_logs(input_file, output_parquet, chunksize=50_000):
2     processed_chunks = []
3     chunk = []
4     with open(input_path, "r", encoding="utf-8") as f:
5         for line in f:
6             if not line.strip():
7                 continue
8             chunk.append(json.loads(line))
9             if len(chunk) >= chunksize:
10                 df_chunk = _transform_chunk(chunk)
11                 processed_chunks.append(df_chunk)
12                 chunk = []
13             if chunk:
14                 df_chunk = _transform_chunk(chunk)
15                 processed_chunks.append(df_chunk)
16     full_df = pd.concat(processed_chunks, ignore_index=True)
17     full_df.to_parquet(output_path, index=False)

```

Listing 3.9: Conversion de JSONL en Parquet par blocs avec extraction de caractéristiques

Le traitement par blocs (par défaut 50 000 enregistrements par bloc) évite l'épuisement de la mémoire lors du traitement de jeux de données de plusieurs millions d'enregistrements.

3.6.2 Équilibrage du Dataset

Le module **balance_parquet.py** aborde le grave déséquilibre des classes dans CICIDS2017 (où le trafic bénin surpasse largement le trafic d'attaque) par le biais d'un sous-échantillonnage aléatoire :

```

1 def balance_parquet_dataset(input_parquet, output_parquet,
2                             target_column="is_attack", random_state
3                             =42):
4     # First pass: count class frequencies
5     df_labels = parquet_file.read(columns=[target_column]).to_pandas()
6     attack_indices = df_labels[df_labels[target_column] == 1].index
7     benign_indices = df_labels[df_labels[target_column] == 0].index
8
9     # Random under-sampling of majority class
10    sampled_benign = np.random.choice(
11        benign_indices, size=len(attack_indices), replace=False
12    )
13    balanced_indices = np.sort(
14        np.concatenate([attack_indices, sampled_benign])
15    )
16    df_balanced = df_full.iloc[balanced_indices].reset_index(drop=True)
17    df_balanced.to_parquet(output_path, index=False)

```

Listing 3.10: Équilibrage efficace en mémoire du dataset via un sous-échantillonnage aléatoire

La stratégie en deux passes (d’abord lire uniquement la colonne des étiquettes pour calculer les indices, puis lire l’ensemble du dataset avec les indices précalculés) optimise l’utilisation de la mémoire pour les grands jeux de données.

Chapter 4

Entraînement du Modèle & Suivi des Expériences

4.1 Ingénierie des Caractéristiques (Feature Engineering)

Le module `src/ml/features.py` définit le pipeline d'ingénierie des caractéristiques, avec un fort accent sur la prévention des fuites de données (data leakage) — une préoccupation majeure dans les applications ML sensibles à la sécurité.

4.1.1 Sélection des Caractéristiques : Le Sous-ensemble de 33 Caractéristiques

La plateforme utilise un sous-ensemble soigneusement sélectionné de 33 caractéristiques à partir des 79 colonnes originales de CICIDS2017. Les critères de sélection excluent :

- **Les identifiants** : `flow_id`, `source_ip`, `src_ip`, `destination_ip`, `dst_ip`, `source_port`, `src_port`, `timestamp`.
- **Les étiquettes cibles (Target labels)** : `label`, `is_attack`.
- **Les caractéristiques potentiellement fuyantes (leaky)** : Les colonnes qui peuvent encoder des informations sur la cible ou qui sont fortement corrélées avec des signatures spécifiques aux attaques.

```
1 FEATURE_NAMES = [  
2     "flow_duration", "total_fwd_packets",  
3     "total_backward_packets",  
4     "total_length_of_fwd_packets",  
5     "total_length_of_bwd_packets",  
6     "fwd_packet_length_max", "fwd_packet_length_min",  
7     "fwd_packet_length_mean", "fwd_packet_length_std",  
8     "bwd_packet_length_max", "bwd_packet_length_min",  
9     "bwd_packet_length_mean", "bwd_packet_length_std",  
10    "flow_iat_mean", "flow_iat_std",  
11    "flow_iat_max", "flow_iat_min",  
12    "fwd_iat_total", "fwd_iat_mean", "fwd_iat_std",  
13    "bwd_iat_total", "bwd_iat_mean", "bwd_iat_std",
```

```

14     "fin_flag_count", "syn_flag_count",
15     "rst_flag_count", "psh_flag_count", "ack_flag_count",
16     "average_packet_size",
17     "active_mean", "active_std",
18     "idle_mean", "idle_std",
19 ]

```

Listing 4.1: Les 33 noms de caractéristiques nettoyés utilisés pour l'entraînement du modèle

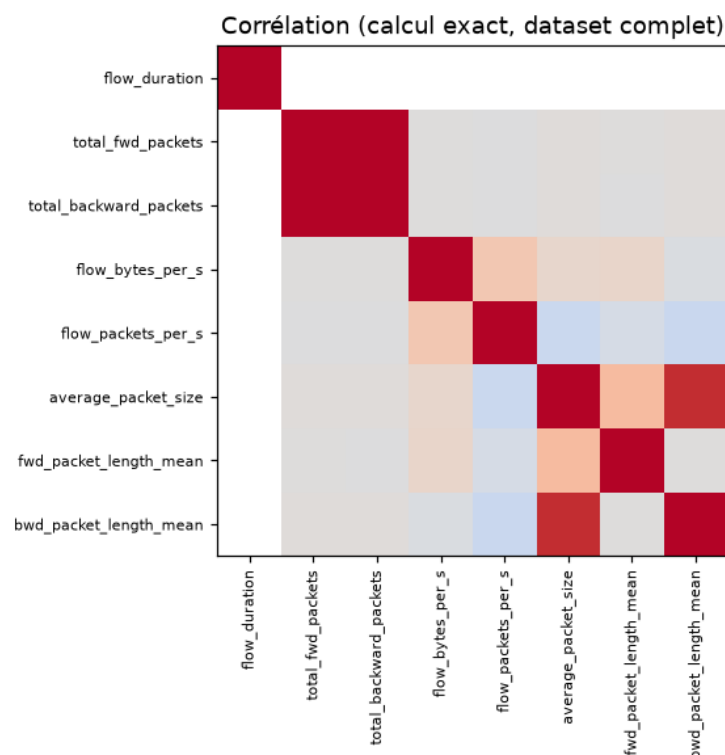


Figure 4.1: Corrélations entre les variables (1) pour l'entraînement du modèle XGBoost.

La matrice de corrélation ci-dessus présente le premier ensemble de relations entre les caractéristiques réseau extraites. Une forte corrélation entre certaines variables indique une multicollinéarité, ce qui peut gonfler artificiellement l'importance perçue de caractéristiques redondantes. En analysant ces corrélations, nous nous assurons que le modèle XGBoost est entraîné sur un sous-ensemble diversifié et informatif de caractéristiques, optimisant ainsi à la fois l'efficacité de calcul et la précision prédictive.

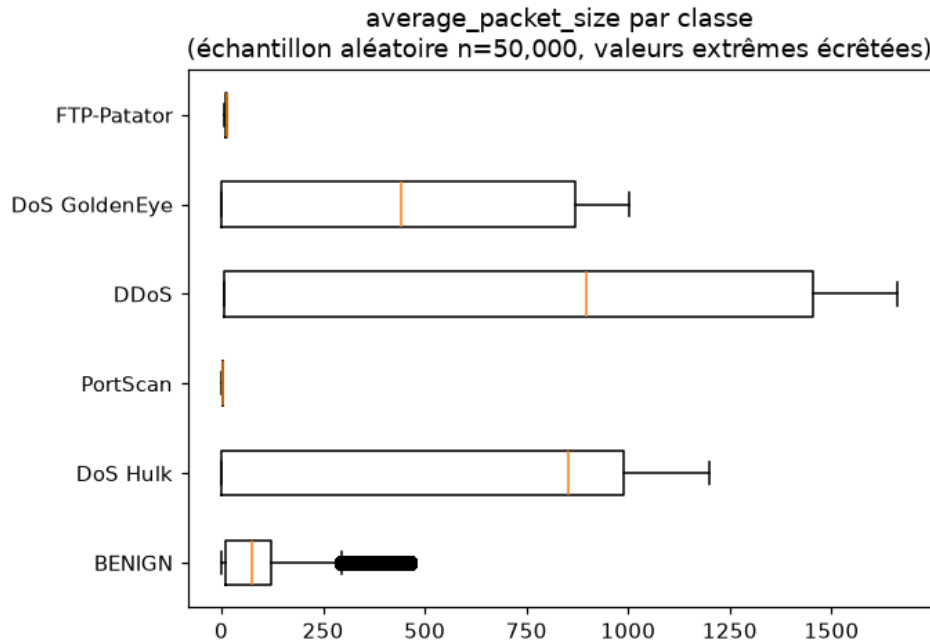


Figure 4.2: Corrélation entre les variables (2) et sélection des variables (Feature Selection).

S'appuyant sur l'analyse de corrélation initiale, la deuxième partie de l'étude de corrélation des caractéristiques affine notre compréhension de la dynamique interne du jeu de données. Cette analyse guide directement le processus de sélection des caractéristiques, garantissant que les 33 caractéristiques choisies sont minimalement corrélées entre elles tout en étant hautement prédictives de la variable cible. La suppression minutieuse des caractéristiques fortement corrélées et redondantes aide à construire un modèle de détection d'anomalies plus robuste et généralisable.

4.1.2 Suppression des Colonnes Sensibles

Une liste de refus (deny-list) explicite garantit que même si des données brutes parviennent à la fonction d'extraction des caractéristiques avec des colonnes inattendues, elles sont supprimées :

```

1 SENSITIVE_COLUMNS_TO_DROP = [
2     "flow_id", "source_ip", "src_ip", "source_port",
3     "src_port", "destination_ip", "dst_ip", "timestamp",
4     "label", "is_attack",
5 ]

```

Listing 4.2: Liste de refus explicite des colonnes sensibles pour la prévention des fuites de données

La fonction `extract_features()` impose un pipeline strict : (1) supprimer toutes les colonnes sensibles, (2) remplir les caractéristiques attendues manquantes avec 0.0, (3)

sélectionner uniquement les 33 caractéristiques autorisées, (4) remplacer les valeurs infinies par NaN, et (5) remplir les NaN restants avec 0.0.

4.1.3 Extraction des Étiquettes (Labels)

La fonction `extract_labels()` gère à la fois les étiquettes multi-classes granulaires et les étiquettes binaires, convertissant le tout en un indicateur d'attaque binaire :

- Si l'étiquette brute est "BENIGN" ou "benign" → 0
- Si le champ `is_attack` est présent, l'utiliser directement
- Sinon, toute étiquette non-bénigne → 1

4.2 Pipeline d'Entraînement

De plus, un notebook d'analyse exploratoire (`Data Analyse/Analyse_EDA_IDS.ipynb`) est inclus pour fournir une vue d'ensemble de la distribution des données et des caractéristiques avant l'entraînement.

Le module `src/ml/train.py` (280 lignes) implémente un pipeline d'entraînement rigoureux, de qualité production. Son architecture reflète plusieurs des meilleures pratiques MLOps.

4.2.1 Sanitisation des Données & Prévention des Fuites

La fonction `sanitize_and_prepare_data()` constitue la première ligne de défense :

```

1 def sanitize_and_prepare_data(df_raw):
2     granular_labels = df_raw["label"] if "label" in df_raw.columns
3     else None
4     y = extract_labels(df_raw)
5     X = extract_features(df_raw)
6
7     # Double vérification : s'assurer qu'aucune colonne sensible n'a
8     # contourné l'extraction
9     leaked = [col for col in SENSITIVE_COLUMNS_TO_DROP if col in X.
10               columns]
11     if leaked:
12         logger.warning(
13             f"Detected leaked columns in feature set! "
14             f"Dropping immediately: {leaked}"
15         )
16     X.drop(columns=leaked, inplace=True, errors="ignore")
17     return X, y, granular_labels

```

Listing 4.3: Sanitisation des données avec double vérification de détection de fuite

Le modèle de "double vérification" (lignes 6–11) est une technique de programmation défensive : bien que `extract_features()` supprime déjà les colonnes sensibles, la fonction d'entraînement vérifie indépendamment qu'aucune n'est passée au travers. Cette approche de défense en profondeur est caractéristique des bases de code soucieuses de la sécurité.

4.2.2 Pondération des Échantillons pour les Classes d'Attaques Rares

Le jeu de données CICIDS2017 présente un grave déséquilibre des classes, non seulement entre le trafic bénin et les attaques, mais aussi parmi les 14 sous-catégories d'attaques. La classe `Heartbleed`, par exemple, peut ne contenir qu'une poignée d'échantillons. Un entraînement naïf sur de telles données amènerait le modèle à ignorer complètement les attaques rares.

La plateforme aborde ce problème avec une **pondération des échantillons atténuée par racine carrée** :

```
1 def compute_dampened_sample_weights(granular_labels):
2     balanced_weights = compute_sample_weight(
3         class_weight="balanced", y=granular_labels
4     )
5     dampened_weights = np.sqrt(balanced_weights)
6     return dampened_weights
```

Listing 4.4: Pondération des échantillons atténuée par racine carrée pour le déséquilibre des classes

La justification de l'atténuation par racine carrée est nuancée :

- La **pondération équilibrée** (`class_weight="balanced"`) attribue des poids inversement proportionnels à la fréquence de la classe : $w_c = \frac{N}{k \cdot n_c}$, où N est le nombre total d'échantillons, k le nombre de classes, et n_c le décompte pour la classe c .
- Pour les classes extrêmement rares, ces poids peuvent être très importants (ex. $> 100\times$), amenant le modèle à sur-apprendre (overfit) sur quelques exemples bruités.
- L'**atténuation par racine carrée** ($w'_c = \sqrt{w_c}$) compresse la plage de poids tout en continuant à surpondérer les classes rares. Cela permet de trouver un équilibre entre ignorer complètement les attaques rares et faire du sur-apprentissage sur celles-ci.

4.2.3 Validation Croisée Stratifiée

La fonction `evaluate_with_cross_validation()` effectue une validation croisée stratifiée à 5 plis sur l'ensemble d'entraînement :

```
1 def evaluate_with_cross_validation(
2     X_train, y_train, sample_weight_train=None,
3     n_splits=5, random_state=42,
4 ):
5     skf = StratifiedKFold(
6         n_splits=n_splits, shuffle=True, random_state=random_state
7     )
8     for fold, (train_idx, val_idx) in enumerate(skf.split(X_train,
9         y_train), 1):
10         X_cv_train, X_cv_val = X_train.iloc[train_idx], X_train.iloc[
11             val_idx]
12         y_cv_train, y_cv_val = y_train[train_idx], y_train[val_idx]
13         sw_cv_train = sample_weight_train[train_idx] if ... else None
14
15     model_cv = xgb.XGBClassifier(
```

```

14         n_estimators=100, max_depth=6, learning_rate=0.1,
15         random_state=random_state, eval_metric="logloss", n_jobs
           =-1,
16     )
17     model_cv.fit(X_cv_train, y_cv_train, sample_weight=sw_cv_train
           )
18     preds = model_cv.predict(X_cv_val)
19     # ... accumulate metrics ...

```

Listing 4.5: Validation croisée stratifiée à 5 plis avec poids des échantillons

Choix de conception clés :

- La **stratification** garantit que chaque pli préserve la distribution originale des classes, ce qui est critique pour les datasets déséquilibrés.
- **Les poids des échantillons sont divisés avec les données** : Le tableau des poids est indexé avec les mêmes indices d'entraînement/validation, empêchant la fuite de poids.
- **Quatre métriques par pli** : L'Exactitude (Accuracy), la Précision, le Rappel (Recall) et le Score F1 fournissent une vue complète de la performance du modèle.

4.2.4 Configuration du Classificateur XGBoost

Le modèle final utilise les hyperparamètres suivants :

Table 4.1: Configuration des hyperparamètres XGBoost.

Paramètre	Valeur	Justification
n_estimators	100	Suffisamment d'arbres pour converger sans sur-apprentissage
max_depth	6	Contrôle la complexité de l'arbre ; empêche la mémorisation
learning_rate	0.1	Taux d'apprentissage standard pour le gradient boosting
eval_metric	logloss	Entropie croisée binaire ; règle de notation appropriée
n_jobs	-1	Utilise tous les cœurs CPU disponibles pour le parallélisme
random_state	42	Garantit la reproductibilité entre les exécutions

4.2.5 Intégration MLflow

Le pipeline d'entraînement s'intègre à MLflow pour un suivi complet des expériences :

```

1  # MLflow Setup
2  tracking_uri = os.environ.get("MLFLOW_TRACKING_URI", "http://mlflow
           :5000")
3  mlflow.set_tracking_uri(tracking_uri)
4  mlflow.set_experiment("network-anomaly-detection")
5
6  # Log parameters
7  mlflow.start_run(run_name="xgboost_weighted_training")
8  mlflow.log_params({
9      "n_estimators": 100, "max_depth": 6,
10     "learning_rate": 0.1, "sample_weighting": True,
11     "features_count": X.shape[1],

```

```

12 })
13
14 # After training
15 mlflow.log_metrics(all_metrics)          # CV + test metrics
16 mlflow.xgboost.log_model(model, artifact_path="model")
17 mlflow.end_run()

```

Listing 4.6: Intégration du suivi d'expériences MLflow

Le modèle de repli progressif (`try/except`) garantit que l'entraînement réussit même si le serveur MLflow est indisponible — une caractéristique de résilience critique pour les environnements de développement et de CI.

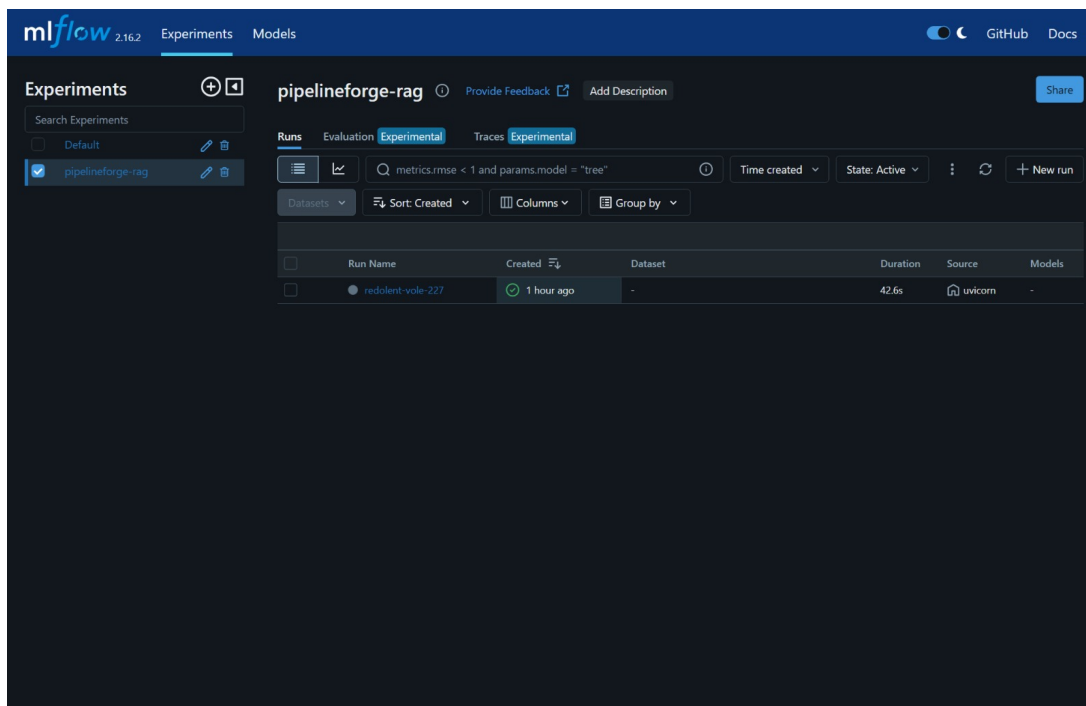


Figure 4.3: Interface de suivi MLflow — liste des exécutions du modèle XGBoost.

La capture d'écran ci-dessus, extraite de `http://<host>:4301`, présente la liste des exécutions enregistrées dans MLflow, avec leurs hyperparamètres, métriques et artefacts associés, illustrant concrètement le suivi des expériences décrit dans cette section.

4.3 Évaluation du Modèle

La stratégie d'évaluation utilise un ensemble de test retenu (20 % des données, stratifié) qui n'est jamais vu lors de la validation croisée. Cinq métriques sont calculées :

- **Exactitude (Accuracy)** : Exactitude globale de la classification.
- **Précision (Precision)** : Proportion d'attaques prédites qui sont de véritables attaques (minimise les fausses alertes).
- **Rappel (Recall)** : Proportion d'attaques réelles qui sont détectées (minimise les attaques manquées).

- **Score F1 (F1-Score)** : Moyenne harmonique de la précision et du rappel ; la principale cible d'optimisation.
- **ROC-AUC** : Aire sous la courbe ROC ; mesure la capacité du modèle à discriminer les classes à travers tous les seuils de probabilité.

Les moyennes de validation croisée et les scores de l'ensemble de test sont enregistrés dans MLflow, permettant la comparaison entre les expériences.

4.4 Module de Prédiction & d'Inférence

Le module `src/ml/predict.py` (196 lignes) fournit l'API d'inférence utilisée à la fois par le service FastAPI et le moteur d'inférence de flux.

4.4.1 Chargement du Modèle avec Repli sur le Registre

La fonction `load_model()` implémente une stratégie de chargement sophistiquée :

```

1  def load_model(model_path=None):
2      model = None
3      # 1. Essayer le Registre MLflow (Stade Production)
4      try:
5          model_uri = "models:/network-anomaly-detector-xgboost/
                        Production"
6          model = mlflow.xgboost.load_model(model_uri)
7      except Exception:
8          # 2. Essayer la dernière version
9          model_uri = "models:/network-anomaly-detector-xgboost/latest"
10         model = mlflow.xgboost.load_model(model_uri)
11
12         # 3. Repli sur l'artefact local
13         if model is None:
14             model_path = get_project_root() / "src/ml/artifacts/model.
                                joblib"
15             model = joblib.load(model_path)
16         return model

```

Listing 4.7: Chargement du modèle avec le registre MLflow et un repli local

La stratégie de chargement à trois niveaux (Production → Latest → Local) offre :

- **Préparation à la production** : En déploiement, les modèles sont promus à travers les stades de MLflow.
- **Commodité de développement** : Les développeurs peuvent travailler sans MLflow en plaçant un fichier `model.joblib` dans le répertoire des artefacts.
- **Compatibilité CI/CD** : Le repli local permet de tester dans des environnements où MLflow n'est pas disponible.

4.4.2 Prédiction Unique et par Lots (Batch)

Les deux fonctions `predict()` et `predict_batch()` suivent le même modèle : convertir l'entrée en un `DataFrame`, aligner les caractéristiques avec les colonnes attendues par le

modèle, exécuter la prédiction, et renvoyer des étiquettes lisibles par l'homme avec des scores de confiance.

```

1 def predict(record, model=None):
2     df_raw = pd.DataFrame([record])
3     df_features = preprocess_features(df_raw)
4
5     # Aligner avec les caractéristiques attendues par le modèle
6     if hasattr(model, "feature_names_in_"):
7         expected_cols = list(model.feature_names_in_)
8         for col in expected_cols:
9             if col not in df_features.columns:
10                df_features[col] = 0.0
11            df_features = df_features[expected_cols]
12
13    pred_code = int(model.predict(df_features)[0])
14    label = "ATTACK" if pred_code == 1 else "BENIGN"
15
16    if hasattr(model, "predict_proba"):
17        proba = model.predict_proba(df_features)[0]
18        confidence = float(proba[pred_code])
19    else:
20        confidence = 1.0
21
22    return label, confidence

```

Listing 4.8: Prédiction sur un enregistrement unique avec alignement des caractéristiques

La logique d'alignement des caractéristiques (lignes 5–10) est critique : elle garantit que la matrice de caractéristiques au moment de l'inférence possède exactement les mêmes colonnes et dans le même ordre que la matrice de caractéristiques lors de l'entraînement, en remplissant toute colonne manquante avec 0.0. Cela empêche l'erreur courante de "désaccord des caractéristiques" (feature mismatch) qui affecte souvent les systèmes ML de production.

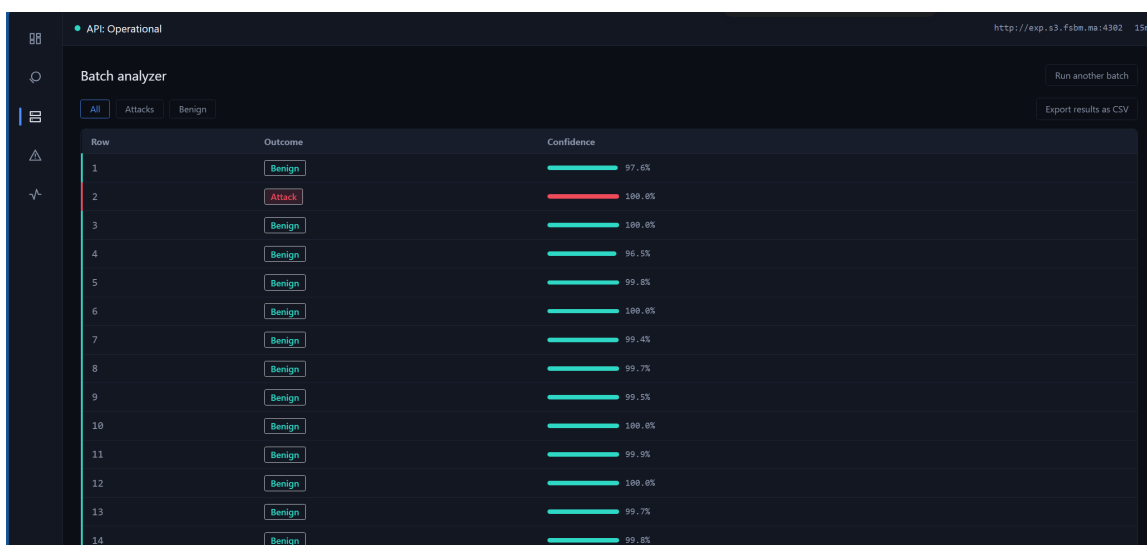


Figure 4.4: Page *Batch analyzer* du frontend, affichant le résultat de la prédiction par lots pour chaque enregistrement d'un fichier CSV téléversé : étiquette (*Benign/Attack*) et score de confiance associé, avec possibilité de filtrer et d'exporter les résultats.

Chapter 5

Intégration de la Sécurité (MLSecOps)

La sécurité n'est pas une réflexion après coup dans la plateforme MLSecOps — elle est tissée dans chaque couche du système. Ce chapitre analyse les contrôles de sécurité mis en œuvre dans la base de code, le pipeline CI/CD, la conteneurisation et la manipulation des données.

5.1 Qualité du Code & Analyse Statique

5.1.1 Crochets de Pré-Validation (Pre-Commit Hooks)

Le fichier `.pre-commit-config.yaml` définit des vérifications automatisées de la qualité du code qui s'exécutent avant chaque validation (commit) Git :

```
1 repos:
2   - repo: https://github.com/pre-commit/pre-commit-hooks
3     rev: v4.4.0
4     hooks:
5       - id: trailing-whitespace
6       - id: end-of-file-fixer
7       - id: check-yaml
8       - id: check-added-large-files
9   - repo: https://github.com/psf/black
10     rev: 23.7.0
11     hooks:
12       - id: black
```

Listing 5.1: Configuration des crochets de pré-validation

Chaque crochet remplit un objectif spécifique en matière de sécurité et de qualité :

Table 5.1: Crochets de pré-validation et leurs implications en matière de sécurité.

Crochet (Hook)	Objectif Sécurité / Qualité
<code>trailing-whitespace</code>	Empêche le bruit dans les diffs qui masque les modifications malveillantes lors de la révision du code
<code>end-of-file-fixer</code>	Garantit des fins de fichier cohérentes ; empêche les erreurs d'analyse dans la CI
<code>check-yaml</code>	Valide la syntaxe YAML (Docker Compose, dbt, configurations CI) ; empêche les mauvaises configurations d'exécution qui pourraient exposer des services
<code>check-added-large-files</code>	Empêche la validation accidentelle de gros fichiers de données, de poids de modèles ou d'identifiants
<code>black</code>	Impose un formatage cohérent ; facilite la révision du code et réduit la surface d'offuscation « basée sur le style » du code malveillant

5.1.2 Analyse Lint avec Flake8

Le pipeline CI exécute flake8 en deux passes :

1. **Erreurs critiques uniquement** : `-select=E9,F63,F7,F82` détecte les erreurs de syntaxe, les problèmes d'assertion, les noms indéfinis et d'autres défauts critiques.
2. **Analyse complète** : `-max-complexity=10 -max-line-length=127` impose des limites de complexité cyclomatique et de longueur de ligne, avec `-exit-zero` pour signaler sans échouer (mode consultatif).

La limite de complexité de 10 est significative : les fonctions dont la complexité cyclomatique dépasse ce seuil sont statistiquement plus susceptibles de contenir des bogues. Dans un contexte de sécurité, les fonctions complexes sont plus difficiles à auditer pour trouver des vulnérabilités.

5.2 Pipeline CI/CD

Le fichier `.github/workflows/ci.yml` définit un pipeline CI complet avec deux tâches (jobs) parallèles :

```

1 name: CI
2 on:
3   push:
4     branches: [main, develop]
5   pull_request:
6     branches: [main, develop]
7
8 jobs:
9   backend:
10    runs-on: ubuntu-latest
11    steps:
12      - uses: actions/checkout@v3
13      - uses: actions/setup-python@v4
14        with:
```

```

15     python-version: '3.10'
16   - name: Install dependencies
17     run: |
18       pip install flake8 pytest pre-commit
19       pip install -r requirements.txt
20   - name: Run Pre-commit Hooks
21     run: pre-commit run --all-files
22   - name: Lint with flake8
23     run: |
24       flake8 . --count --select=E9,F63,F7,F82 --show-source
25       flake8 . --count --exit-zero --max-complexity=10
26   - name: Run Pytest
27     run: pytest
28   - name: Build Docker Image
29     run: docker build -t mlsecops-app:${{ github.sha }} .
30
31 frontend:
32   runs-on: ubuntu-latest
33   defaults:
34     run:
35       working-directory: frontend
36   steps:
37     - uses: actions/checkout@v3
38     - uses: actions/setup-node@v4
39       with:
40         node-version: '20'
41     - run: npm ci
42     - run: npm run build

```

Listing 5.2: Pipeline CI GitHub Actions (tâche backend)

Le pipeline CI impose un **processus de qualité à quatre portes (gates)** :

1. **Pré-validation (Pre-commit)** : Formatage du code et hygiène de base.
2. **Flake8** : Analyse statique pour le code Python.
3. **Pytest** : Exécution automatisée de la suite de tests.
4. **Construction Docker** : La construction de l'image valide le Dockerfile et l'installation des dépendances.

L'image Docker est étiquetée avec le SHA Git (`${{ github.sha }}`), fournissant des builds immuables et traçables qui peuvent être audités après le déploiement.

La tâche frontend valide séparément la compilation TypeScript et le regroupement (bundling) Vite, garantissant que les déploiements frontend n'introduisent pas d'erreurs d'exécution.

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
c16979dffe6	mlsecops-platform-frontend	"/docker-entrypoint..."	4 days ago	Up About a minute	0.0.0.0:4309->80/tcp, [::]:4309->80/tcp	frontend_dashboard
f02d282eff93	mlsecops-platform-dagster-daemon	"dagster-daemon run"	4 days ago	Up About a minute		dagster_daemon
3083535a2cd4	mlsecops-platform-dagster	"dagster-webserver ..."	4 days ago	Up About a minute	0.0.0.0:4308->3000/tcp, [::]:4308->3000/tcp	dagster_orchestrator
c397385ef381	mlsecops-platform-fastapi-app	"uvicorn app.main:ap..."	4 days ago	Up About a minute	0.0.0.0:4302->8000/tcp, [::]:4302->8000/tcp	fastapi_anomaly
8a445762fc07	mlsecops-platform-mlflow	"mlflow server --hos..."	4 days ago	Up About a minute	0.0.0.0:4301->5000/tcp, [::]:4301->5000/tcp	mlflow_server
6e51f3a4fecb	prom/prometheus:latest	"/bin/prometheus --c..."	4 days ago	Up About a minute	0.0.0.0:4303->9090/tcp, [::]:4303->9090/tcp	prometheus
a61bb7690848	grafana/grafana:latest	"/run.sh"	4 days ago	Up About a minute	0.0.0.0:4304->3000/tcp, [::]:4304->3000/tcp	grafana
f0c62c58da3a	redpandadata/redpanda:v24.1.1	"/entrypoint.sh redp..."	4 days ago	Up About a minute	9644/tcp, 0.0.0.0:4305->8081/tcp, [::]:4305->8081/tcp, 0.0.0.0:4306->8082/tcp, [::]:4306->8082/tcp, 0.0.0.0:4307->9092/tcp, [::]:4307->9092/tcp, 0.0.0.0:4308->29092/tcp, [::]:4308->29092/tcp	redpanda

Figure 5.1: Sortie de `docker compose ps` — les huit services de la plateforme à l'état Up.

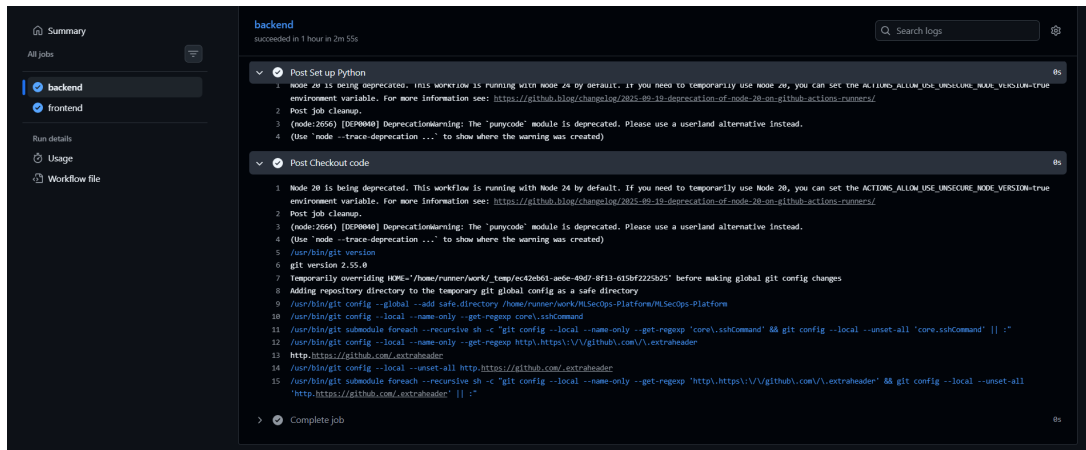


Figure 5.2: Pipeline GitHub Actions — exécution réussie (pre-commit, flake8, pytest, build Docker).

Ces captures confirment que les huit services conteneurisés sont opérationnels simultanément et que le pipeline CI/CD s'exécute intégralement avec succès à chaque push, validant les quatre portes de qualité décrites plus haut.

5.3 Stratégie de Test

5.3.1 Configuration de Pytest

Le fichier `pytest.ini` configure pytest pour découvrir les tests dans le répertoire `tests/` avec la racine du projet sur le chemin Python :

```
1 [pytest]
2 pythonpath = .
3 testpaths = tests
```

Listing 5.3: Configuration de `pytest.ini`

5.3.2 Tests de Qualité des Données (537 Lignes)

Le fichier `tests/test_data_quality.py` (537 lignes) fournit une couverture exhaustive du framework de validation de la qualité des données. Il est organisé en quatre classes de test :

TestValidateRawCSV (11 tests) : Couvre les données valides, les DataFrames vides, les nombres de colonnes incorrects, les colonnes critiques manquantes, les taux élevés de valeurs nulles, les lignes complètement vides, les plages de ports invalides, les étiquettes (labels) inconnues, les valeurs infinies, les valeurs négatives et les valeurs de port aux limites.

TestValidateCleanedRecords (5 tests) : Couvre les enregistrements valides, les enregistrements vides, les clés critiques manquantes, les clés non normalisées, les ports invalides et les enregistrements corrompus avec des champs insuffisants.

TestValidateInferenceOutput (8 tests) : Couvre l'inférence valide, zéro anomalie, les métadonnées ML manquantes, la confiance hors plage, la confiance négative, la version de modèle vide, la confiance sous le seuil et les nombres d'anomalies non plausibles.

TestQualityReportAndError (3 tests) : Couvre le contenu du résumé du rapport, la sérialisation du rapport et la propagation des erreurs.

Les fixtures de test utilisent des usines (factories) de données soigneusement conçues :

```

1  @pytest.fixture
2  def good_raw_df():
3      """Create a minimal valid CICIDS2017-style DataFrame with 79
4          columns."""
5      data = {
6          "Destination Port": [80],
7          "Flow Duration": [1000],
8          "Total Fwd Packets": [10],
9          # ... 76 more columns ...
10         "Label": ["BENIGN"],
11     }
12     df = pd.DataFrame(data)
13     # Real CSV has "Fwd Header Length" twice; replicate this quirk
14     fwd_header_dup = df["Fwd Header Length"].copy()
15     df.insert(idx, "Fwd Header Length", fwd_header_dup,
16             allow_duplicates=True)
17     return df

```

Listing 5.4: Fixture de test créant un DataFrame CICIDS2017 valide à 79 colonnes

Notez l'attention portée aux détails : la fixture reproduit la particularité du vrai CSV CICIDS2017 d'avoir une colonne **Fwd Header Length** en double, garantissant que les tests valident par rapport au schéma de données réel.

5.4 Sécurité des Conteneurs

5.4.1 Exécution Sans Privilèges Root

Les deux Dockerfiles créent un utilisateur dédié **appuser** et basculent vers celui-ci avant d'exécuter la commande du conteneur :

```

1  RUN useradd -m appuser && chown -R appuser:appuser /app
2  USER appuser

```

Listing 5.5: Création d'un utilisateur non-root dans le Dockerfile

L'exécution des conteneurs en tant qu'utilisateurs non-root atténue l'impact des vulnérabilités d'évasion de conteneur. Si un attaquant exploite une vulnérabilité dans l'application FastAPI (ex., lecture de fichier arbitraire), il ne peut pas accéder aux fichiers du système hôte ni modifier les propres binaires du conteneur.

5.4.2 Images de Base Allégées (Slim)

L'utilisation de `python:3.10-slim` réduit la surface d'attaque en excluant :

- Les outils de construction et les compilateurs (dans le Dockerfile racine)
- La documentation système et les pages de manuel
- Les bibliothèques système inutiles

5.4.3 Isolation du Réseau Docker

Tous les services communiquent sur un réseau de type pont (bridge) isolé (`mlops_network`) :

```
1 networks:
2   mlops_network:
3     driver: bridge
```

Listing 5.6: Isolation du réseau Docker

Cela empêche l'accès externe à la communication inter-services et garantit que les services ne peuvent se joindre les uns les autres qu'à travers les noms DNS définis par Docker.

5.5 Prévention des Fuites de Données

La fuite de données (data leakage) — où des informations provenant de la variable cible ou de l'ensemble de test influencent par inadvertance l'entraînement — est une préoccupation critique dans le ML de sécurité. La plateforme met en œuvre plusieurs couches de défense :

1. **Liste blanche (Allow-list) des caractéristiques** : Seules 33 caractéristiques pré-approuvées sont utilisées, définies dans `FEATURE_NAMES`.
2. **Liste de refus des colonnes sensibles** : `SENSITIVE_COLUMNS_TO_DROP` nomme explicitement 10 colonnes à supprimer.
3. **Double vérification lors de l'entraînement** : `sanitize_and_prepare_data()` vérifie indépendamment qu'aucune colonne sensible n'a survécu à l'extraction des caractéristiques.
4. **Séparation Entraînement/Test avant tout ajustement (fitting)** : La division utilise `stratify=y` pour maintenir la distribution des classes, et les poids des échantillons sont divisés simultanément pour éviter la fuite des poids.

5.6 Sécurité de l'API

L'application FastAPI implémente plusieurs contrôles de sécurité au niveau de l'API :

- **Restriction CORS** : Seules les origines `localhost:5173` et `localhost:3000` sont autorisées, ce qui empêche les attaques cross-origin à partir de sites web arbitraires.
- **Validation des entrées** : Le modèle `FlowItem` de Pydantic impose que les 33 champs requis soient présents avec les types corrects. Les requêtes invalides sont rejetées avec des codes d'état 422.

- **Sondes d'état (Health probes)** : Le point de terminaison (endpoint) `/health` signale si le modèle est chargé, permettant aux orchestrateurs (Kubernetes, vérifications d'état Docker) d'acheminer le trafic uniquement vers des instances saines.
- **Gestion gracieuse des erreurs** : Tous les endpoints de prédiction interceptent les exceptions, les consignent (log) avec `exc_info=True` pour le débogage, et renvoient des messages d'erreur sanitaires pour empêcher la fuite d'informations.

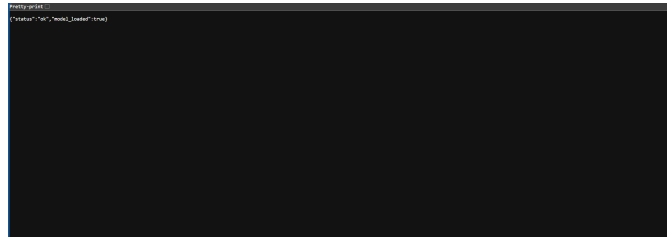


Figure 5.3: Réponse JSON du point de terminaison `/health`, confirmant que le service est opérationnel (`"status": "ok"`) et que le modèle XGBoost a bien été chargé en mémoire (`"model_loaded": true`).

Chapter 6

Orchestration avec Dagster

6.1 Conception de Pipeline Centrée sur les Actifs (Assets)

Le paradigme des *actifs définis par logiciel* (software-defined assets) de Dagster représente un changement fondamental par rapport à l'approche traditionnelle centrée sur les tâches utilisée par des outils comme Airflow. Dans la plateforme MLSecOps, le pipeline est défini en termes d'*artefacts de données* (ce qui est produit) plutôt qu'en termes de *tâches* (ce qui est exécuté). Cette approche offre plusieurs avantages :

- **Traçabilité des données (Data lineage)** : Les dépendances entre les actifs sont explicites, ce qui permet la visualisation automatique du graphe de flux de données.
- **Idempotence** : Les actifs peuvent être re-matérialisés indépendamment sans avoir à réexécuter l'ensemble du pipeline.
- **Observabilité** : L'interface utilisateur de Dagster affiche les métadonnées des actifs (nombre d'événements, chemins de fichiers, métriques) directement à côté du graphe d'exécution.

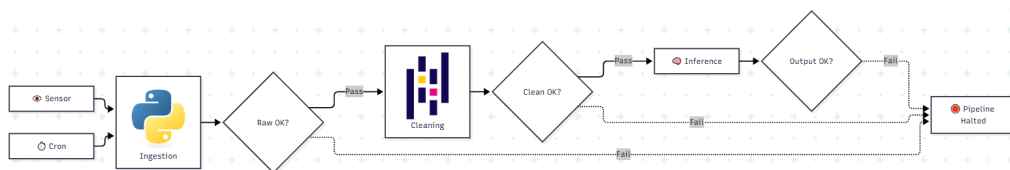


Figure 6.1: Graphe orienté acyclique (DAG) du pipeline Dagster.

Ce diagramme représente le graphe de dépendances entre les trois actifs Dagster (`streaming_ingestion_asset`, `streaming_cleaning_asset`, `model_inference_asset`), déclenchés soit par le capteur `csv_streaming_sensor` (intervalle de 30 secondes), soit par la planification horaire `hourly_streaming_schedule`. Chaque actif est suivi d'une porte de qualité (`validate_raw_csv`, `validate_cleaned_records`, `validate_inference_output`) qui peut interrompre l'exécution du pipeline en cas d'échec, matérialisant le principe de sémantique d'arrêt décrit dans cette section.

6.2 Actifs du Pipeline (Pipeline Assets)

La plateforme définit trois actifs principaux dans le fichier `src/orchestration/assets.py` :

6.2.1 Actif d'Ingestion de Flux (Streaming Ingestion Asset)

```

1  @asset(
2      group_name="streaming_pipeline",
3      compute_kind="python",
4      description="Simulates real-time network traffic by "
5                  "publishing local CSVs to Redpanda (raw-logs).",
6  )
7  def streaming_ingestion_asset(context: AssetExecutionContext):
8      # PORTE DE QUALITÉ DES DONNÉES : Valider les fichiers CSV bruts
9        avant l'ingestion
10     for filepath in csv_files:
11         df = pd.read_csv(filepath, keep_default_na=False, nrows=10000)
12         numeric_cols = df.select_dtypes(include=[np.number]).columns
13         df[numeric_cols] = df[numeric_cols].replace(
14             [np.inf, -np.inf], np.nan
15         )
16         try:
17             report = validate_raw_csv(df)
18             context.log.info(f"Raw data quality PASSED for {filepath}")
19         except DataQualityError as e:
20             context.log.error(f"Raw data quality FAILED")
21             raise # Arrêter le pipeline
22
23     total_published = run_producer(
24         batch_size=_read_batch_size(), max_records=target
25     )
26     context.add_output_metadata({
27         "events_published": MetadataValue.int(events_published),
28     })
29     return events_published

```

Listing 6.1: Actif Dagster pour l'ingestion de flux avec porte de qualité

La porte de qualité s'exécute *avant* que les données ne soient publiées dans Kafka. Si un fichier CSV échoue à la validation, l'ensemble du pipeline s'arrête en fournissant des informations de diagnostic complètes. Il s'agit d'une fonctionnalité de sécurité essentielle : elle empêche l'entrée de données empoisonnées ou corrompues dans le réseau de diffusion (streaming backbone).

6.2.2 Actif de Nettoyage de Flux (Streaming Cleaning Asset)

L'actif de nettoyage dépend de l'actif d'ingestion (`deps=[streaming_ingestion_asset]`) et exécute le nettoyeur de flux suivi d'une validation de qualité de l'export nettoyé :

```

1  @asset(
2      deps=[streaming_ingestion_asset],
3      group_name="streaming_pipeline",

```

```

4     compute_kind="python",
5 )
6 def streaming_cleaning_asset(context):
7     cleaned = run_cleaner(max_messages=target)
8     # Porte de qualité : valider l'export nettoyé
9     report = validate_cleaned_records(records)
10    return cleaned

```

Listing 6.2: Déclaration de dépendance d'actif pour le nettoyage

6.2.3 Actif d'Inférence de Modèle (Model Inference Asset)

L'actif d'inférence s'exécute après le nettoyage et applique le modèle ML au flux de données nettoyé, avec une validation de qualité post-inférence :

```

1 @asset(
2     deps=[streaming_cleaning_asset],
3     group_name="streaming_pipeline",
4     compute_kind="machine_learning",
5 )
6 def model_inference_asset(context):
7     anomalies = run_inference(max_messages=target)
8     # Porte de qualité : valider la sortie d'inférence
9     report = validate_inference_output(records, total_processed=target)
10    return anomalies

```

Listing 6.3: Actif d'inférence de modèle avec validation de qualité

6.3 Capteurs et Planifications (Sensors and Schedules)

6.3.1 Capteur du Système de Fichiers (File-System Sensor)

Le `csv_streaming_sensor` surveille le répertoire `data/raw/` pour détecter les fichiers CSV nouveaux ou modifiés :

```

1 @sensor(
2     name="csv_streaming_sensor",
3     job=streaming_pipeline_job,
4     minimum_interval_seconds=30,
5 )
6 def csv_streaming_sensor(context: SensorEvaluationContext):
7     csv_files = glob.glob(os.path.join(data_dir, "*.csv"))
8     current_state = {}
9     for filepath in csv_files:
10        stats = os.stat(filepath)
11        current_state[os.path.basename(filepath)] = \
12            f"{stats.st_mtime}_{stats.st_size}"
13
14    last_state = json.loads(context.cursor or "{}")
15    if current_state != last_state:
16        context.update_cursor(json.dumps(current_state))

```

```

17         yield RunRequest(
18             run_key=f"streaming_run_{datetime.now().timestamp()}",
19         )

```

Listing 6.4: Capteur Dagster pour le déclenchement automatique du pipeline

Le capteur utilise une **comparaison d'état basée sur un curseur** : il sérialise les temps de modification et les tailles actuels des fichiers sous forme de chaîne JSON, les compare au curseur précédent et ne déclenche une nouvelle exécution que si des changements sont détectés. Cela évite les exécutions en double et garantit une sémantique de traitement "exactement une fois".

6.3.2 Planification de Sécurité Horaire (Hourly Safety Schedule)

En tant que mécanisme de secours, une planification cron horaire garantit que le pipeline s'exécute au moins une fois par heure :

```

1 hourly_streaming_schedule = ScheduleDefinition(
2     name="hourly_streaming_schedule",
3     job=streaming_pipeline_job,
4     cron_schedule="0 * * * *",
5 )

```

Listing 6.5: Planification de sécurité horaire

6.4 Assemblage des Définitions (Definitions Assembly)

Tous les actifs, tâches, planifications et capteurs sont assemblés dans un seul objet `Definitions` :

```

1 defs = Definitions(
2     assets=[
3         streaming_ingestion_asset,
4         streaming_cleaning_asset,
5         model_inference_asset,
6     ],
7     jobs=[streaming_pipeline_job],
8     schedules=[hourly_streaming_schedule],
9     sensors=[csv_streaming_sensor],
10 )

```

Listing 6.6: Assemblage des définitions Dagster

La configuration de l'espace de travail Dagster (`workspace.yaml`) pointe vers ce module de définitions :

```

1 load_from:
2   - python_file:
3     relative_path: src/orchestration/definitions.py

```

Listing 6.7: Configuration de l'espace de travail Dagster

Chapter 7

Déploiement & Surveillance

7.1 Orchestration avec Docker Compose

L'ensemble de la plateforme est déployé sous la forme d'une pile Docker Compose unique, définie dans `docker-compose.yml`. Cette section analyse les définitions de services, la gestion des volumes et la topologie du réseau.

7.1.1 Définitions de Services

Les huit services sont organisés en quatre groupes fonctionnels :

Couche d'Orchestration (Serveur web Dagster + démon) :

```
1  dagster:
2    user: root
3    build:
4      context: .
5      dockerfile: src/Dockerfile
6    container_name: dagster_orchestrator
7    ports:
8      - "4300:3000"
9    volumes:
10     - ./app
11     - dagster_home:/app/dagster_home
12     - mlflow_data:/app/mlruns
13    depends_on:
14     - redpanda
15    environment:
16     - PYTHONPATH=/app
17     - DAGSTER_HOME=/app/dagster_home
18     - KAFKA_BOOTSTRAP_SERVERS=redpanda:9092
19     - STREAM_MAX_RECORDS=40000000
20     - PRODUCER_BATCH_SIZE=5000
21     - MLFLOW_TRACKING_URI=http://mlflow:5000
22    command: ["dagster-webserver", "-h", "0.0.0.0", "-p", "3000"]
```

Listing 7.1: Configuration du service Dagster avec variables d'environnement

Le service Dagster s'exécute en tant que root (outrepassant l'USER `appuser` du Dockerfile)

car il a besoin d'écrire sur les volumes gérés par Docker. Dans un déploiement de production, cela serait atténué avec des permissions de volume appropriées.

Couche de Flux (Streaming) (Redpanda) : Le service Redpanda est configuré avec des limites de ressources (`-memory 1G`) et des doubles écouteurs Kafka pour un accès intra-conteneur et depuis l'hôte.

Couche ML (MLflow + FastAPI) : Le service FastAPI dépend de MLflow (`depends_on: [mlflow]`) pour garantir que le serveur de suivi des expériences est disponible au chargement du modèle. La commande MLflow configure le serveur avec `-allowed-hosts *` pour une flexibilité de développement.

Couche de Surveillance (Prometheus + Grafana) : Les deux services de surveillance utilisent des volumes Docker persistants pour conserver les données lors des redémarrages de conteneurs.

7.1.2 Gestion des Volumes

Cinq volumes Docker nommés assurent la persistance des données :

Table 7.1: Volumes Docker et leurs objectifs.

Volume	Objectif
<code>redpanda_data</code>	Données de topic Kafka et décalages (offsets) des consommateurs
<code>dagster_home</code>	Historique d'exécution Dagster, journaux d'événements et planifications
<code>mlflow_data</code>	Artefacts d'expérience MLflow et fichiers de modèle
<code>prometheus_data</code>	Base de données de séries chronologiques (TSDB) de Prometheus
<code>grafana_data</code>	Tableaux de bord Grafana, sources de données et configuration utilisateur

7.1.3 Topologie du Réseau

Tous les services sont connectés au réseau de pont `mlops_network`, permettant la découverte de services basée sur DNS. Par exemple, le service FastAPI accède à MLflow via `http://mlflow:5000`, tandis que Dagster se connecte à Redpanda via `redpanda:9092`.

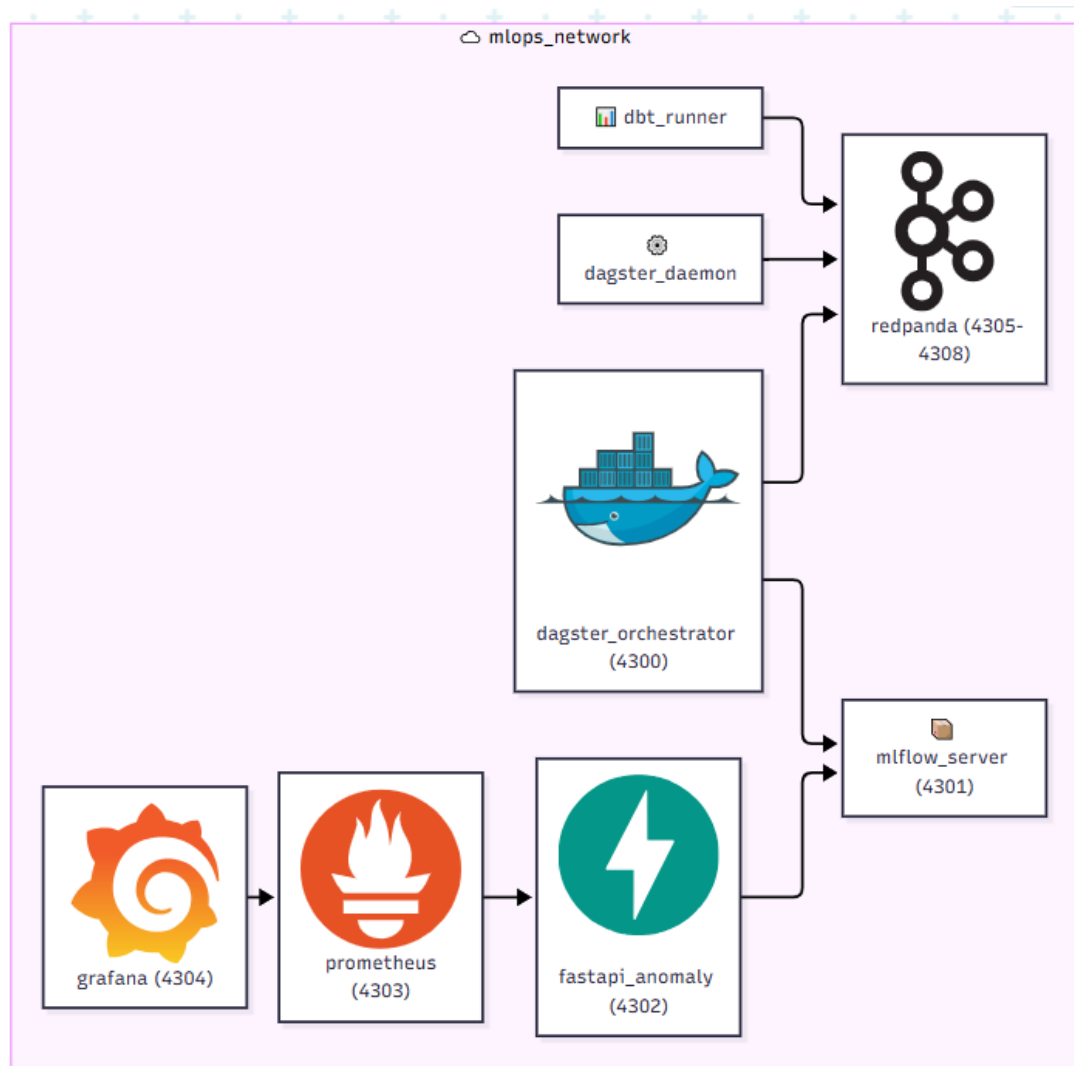


Figure 7.1: Topologie du réseau Docker Compose (`mlops_network`).

Ce diagramme illustre la manière dont les huit services conteneurisés de la plateforme (`dagster_orchestrator`, `dagster_daemon`, `dbt_runner`, `redpanda`, `mlflow_server`, `fastapi_anomaly`, `prometheus`, `grafana`) communiquent au sein du réseau bridge isolé `mlops_network`, ainsi que les ports exposés à l'hôte (4300–4308, remappés pour le déploiement Komodo). Il met en évidence les dépendances de communication interservices, notamment entre le service d'inférence FastAPI et le serveur de suivi MLflow, et entre Grafana et Prometheus pour la visualisation des métriques.

7.2 Couche de Service FastAPI (Serving Layer)

7.2.1 Cycle de Vie de l'Application

L'application FastAPI utilise un gestionnaire de contexte asynchrone pour gérer le cycle de vie du modèle :

```
1 @asynccontextmanager
2 async def lifespan(app: FastAPI):
```

```

3     """Load the trained model once at startup; release on shutdown."""
4     model = load_model("src/ml/artifacts/model.joblib")
5     app.state.model = model
6     yield
7     app.state.model = None
8
9 app = FastAPI(
10     title="Network Anomaly Detection API",
11     description="Real-time network security inference engine "
12                 "built on FastAPI & XGBoost",
13     version="1.0.0",
14     lifespan=lifespan,
15 )

```

Listing 7.2: Cycle de vie de l'application FastAPI avec chargement du modèle

Ce modèle (pattern) garantit que :

- Le modèle est chargé *une seule fois* au démarrage, et non à chaque requête.
- La référence du modèle est stockée dans `app.state`, ce qui la rend accessible à tous les gestionnaires de points de terminaison (endpoints).
- À l'arrêt, le modèle est explicitement défini sur `None`, permettant le ramassage des miettes (garbage collection).

7.2.2 Instrumentation Prometheus

Trois métriques personnalisées sont définies au niveau du module et mises à jour dans chaque point de terminaison de prédiction :

```

1 PREDICTIONS_TOTAL = Counter(
2     "model_predictions_total",
3     "Total number of predictions made by the model",
4     ["model_version"],
5 )
6 ANOMALIES_DETECTED = Counter(
7     "anomalies_detected_total",
8     "Total number of network anomalies detected",
9 )
10 MODEL_INFERENCE_TIME = Histogram(
11     "model_inference_seconds",
12     "Time spent running model inference",
13 )

```

Listing 7.3: Instrumentation des métriques Prometheus

Le point de terminaison `/metrics` les expose dans le format d'exposition Prometheus :

```

1 @app.get("/metrics")
2 async def get_metrics():
3     return Response(
4         content=generate_latest(),
5         media_type=CONTENT_TYPE_LATEST,
6     )

```

Listing 7.4: Point de terminaison de collecte (scrape) Prometheus

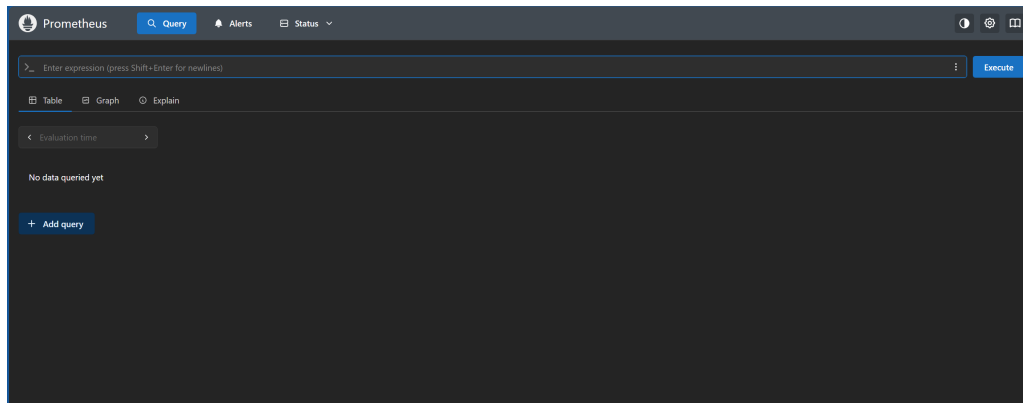


Figure 7.2: Interface de requêtage (*Query*) de Prometheus, utilisée pour interroger de manière ad hoc les métriques exposées par le point de terminaison `/metrics` de FastAPI (par exemple `model_predictions_total` ou `anomalies_detected_total`).

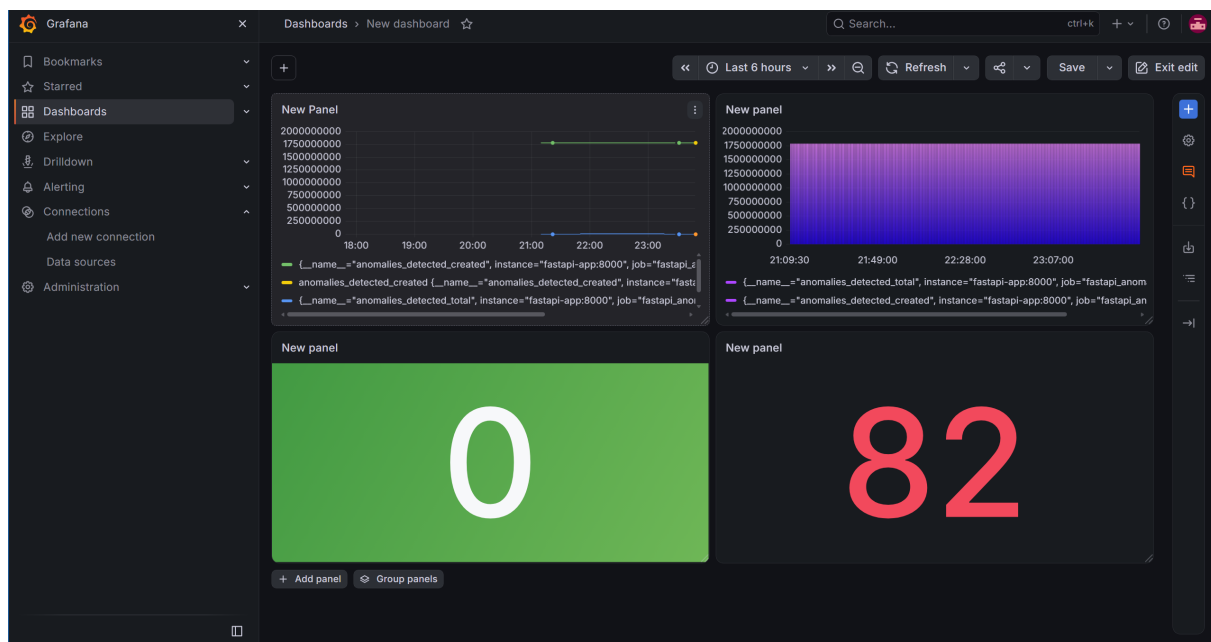


Figure 7.3: Tableau de bord Grafana construit sur la source de données Prometheus, combinant une série temporelle des compteurs `anomalies_detected_total/anomalies_detected_created`, un histogramme de la charge de requêtes, et deux panneaux de type jauge (*stat*) affichant respectivement 0 anomalie récente et 82 prédictions cumulées.

7.2.3 Validation de Schéma Pydantic

Le modèle Pydantic `FlowItem` définit les 33 caractéristiques requises du flux réseau avec des descriptions de champs complètes :

```
1 class FlowItem(BaseModel):
2     model_config = {"extra": "allow"}
3
4     flow_duration: float = Field(
5         ..., description="Duration of the flow in microseconds"
```

```

6     )
7     total_fwd_packets: int = Field(
8         ..., description="Total packets in the forward direction"
9     )
10    total_backward_packets: int = Field(
11        ..., description="Total packets in the backward direction"
12    )
13    # ... 30 more fields with descriptions ...
14
15    class BatchPredictRequest(BaseModel):
16        flows: List[FlowItem] = Field(
17            ..., description="List of network flows to predict in bulk"
18        )

```

Listing 7.5: Schéma Pydantic pour la validation du flux réseau (extrait)

La configuration `extra = "allow"` permet des champs supplémentaires au-delà des 33 requis, assurant une compatibilité ascendante avec les futurs ajouts de caractéristiques.

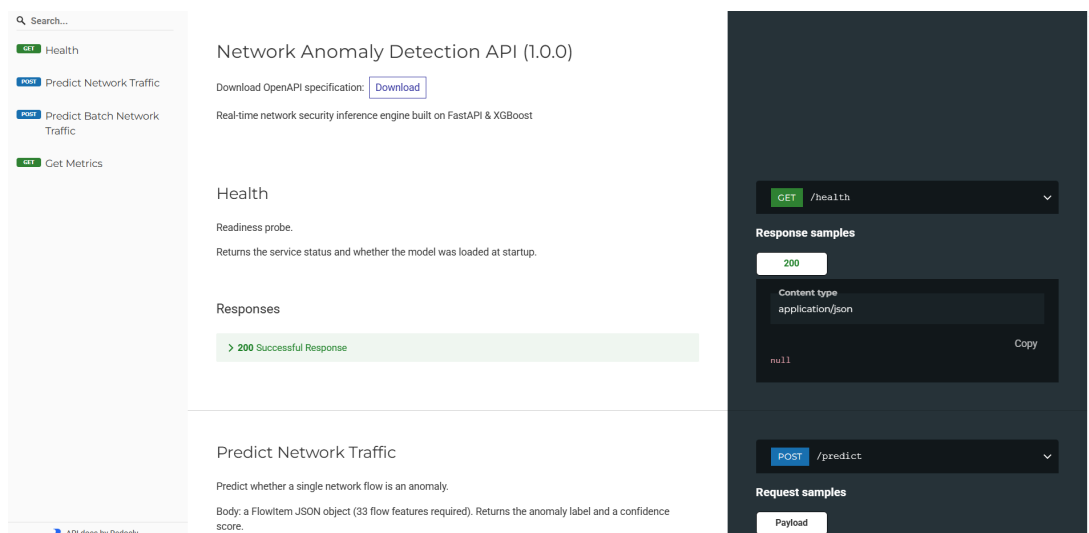


Figure 7.4: Documentation interactive FastAPI (Swagger UI) — endpoints `/health`, `/predict`, `/predict/batch` et `/metrics`.

Cette capture, extraite du point de terminaison `/docs` (`http://<host>:4302`), présente les contrats d'API générés automatiquement par FastAPI à partir des schémas Pydantic, servant à la fois de documentation et d'interface de test manuel.

7.3 Application Frontend

7.3.1 Architecture

Le frontend suit une architecture classique SPA (Single Page Application) React :

- **Routeur** (`router.tsx`) : React Router v7 avec six routes à l'intérieur d'un composant de mise en page `AppShell`.
- **Contexte** (`SessionProvider`) : Gestion de l'état global de l'application.

- **Couche API** (api/) : Client HTTP typé (`apiRequest<T>()`) avec gestion des erreurs, URL de base configurable, et traitement par lots (batch) par morceaux (chunks).
- **Composants** : Organisés dans des sous-répertoires par fonctionnalité (alerts, batch, charts, inspector, layout, ui).

7.3.2 Architecture du Client API

Le client API du frontend (`client.ts`) implémente plusieurs modèles (patterns) robustes :

```

1 export function getApiBaseUrl(): string {
2   const stored = localStorage.getItem(API_BASE_URL_STORAGE_KEY)
3   if (stored && stored.trim().length > 0) return stored
4   return import.meta.env.VITE_API_BASE_URL || 'http://localhost:8000'
5 }
6
7 export async function apiRequest<T>(<
8   path: string, options: RequestOptions = {}
9 ): Promise<T> {
10   const url = `${base.replace(/\/$/, ' ')}${path}`
11   try {
12     response = await fetch(url, {
13       method: options.method ?? 'GET',
14       headers: options.body
15         ? { 'Content-Type': 'application/json' }
16         : undefined,
17       body: options.body ? JSON.stringify(options.body) : undefined,
18       signal: options.signal,
19     })
20   } catch (err) {
21     if (err instanceof DOMException && err.name === 'AbortError')
22       throw err
23     throw new ApiError(err.message, 0, null)
24   }
25   if (!response.ok) {
26     throw new ApiError('Request failed with ${response.status}',
27       response.status, payload)
28   }
29   return payload as T
30 }

```

Listing 7.6: Client API frontend avec URL de base configurable et gestion des erreurs

Les principaux modèles incluent :

- **URL de base configurable** : Stockée dans le `localStorage`, modifiable via la page Paramètres (Settings), avec solution de repli (fallback) sur la variable d'environnement `Vite`.
- **Support de l'AbortController** : Permet l'annulation de requêtes pour les prédictions par lots (batch) de longue durée.
- **Gestion des erreurs typées** : `ApiError` transporte le code de statut HTTP et le corps de réponse analysé.

7.3.3 Stratégie de Traitement par Lots (Batch Processing)

Le point de terminaison de prédiction par lots découpe les gros téléchargements (uploads) de fichiers CSV en morceaux de 500 enregistrements :

```

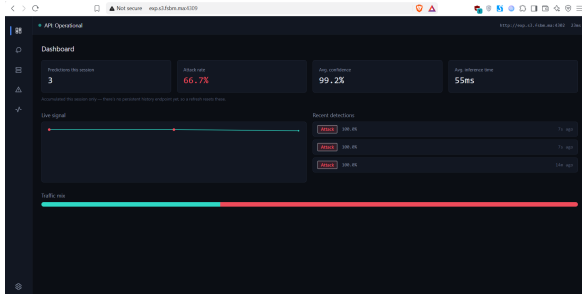
1  const BATCH_CHUNK_SIZE = 500
2
3  export async function predictBatch(
4    flows: FlowItem[],
5    onProgress?: (completed: number, total: number) => void,
6    signal?: AbortSignal,
7  ): Promise<PredictBatchOutcome> {
8    const results: PredictResponse[] = []
9    let failedCount = 0
10   for (let i = 0; i < flows.length; i += BATCH_CHUNK_SIZE) {
11     const chunk = flows.slice(i, i + BATCH_CHUNK_SIZE)
12     try {
13       const response = await apiRequest<BatchPredictResponse>(
14         '/predict/batch', { method: 'POST', body: { flows: chunk },
15         signal }
16       )
17       results.push(...response.predictions)
18     } catch (err) {
19       if (err instanceof DOMException && err.name === 'AbortError')
20         throw err
21       failedCount += chunk.length
22       // Remplir avec des résultats factices (placeholders) pour les
23       // morceaux ayant échoué
24       for (let j = 0; j < chunk.length; j++) {
25         results.push({ is_anomaly: 'BENIGN', confidence: 0 })
26       }
27     }
28     onProgress?.(Math.min(i + BATCH_CHUNK_SIZE, flows.length), flows.
29       length)
30   }
31   return { results, failedCount }
32 }

```

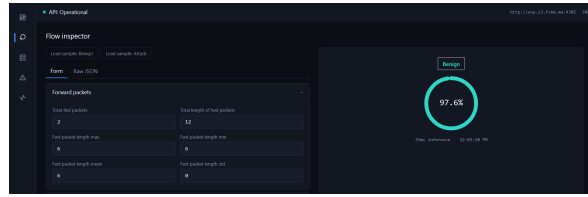
Listing 7.7: Prédiction par lots basée sur des morceaux (chunks) avec suivi de la progression

Cette approche permet :

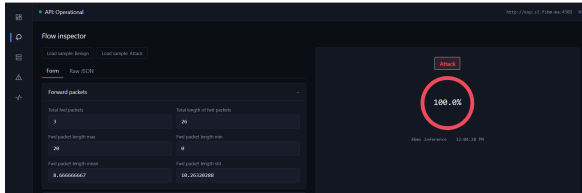
- **Rapport de progression** : Le rappel (callback) `onProgress` permet à l'UI d'afficher une barre de progression.
- **Tolérance aux pannes partielles** : Si un morceau échoue, les morceaux restants sont tout de même traités.
- **Support de l'annulation** : L'`AbortSignal` permet une annulation initiée par l'utilisateur.



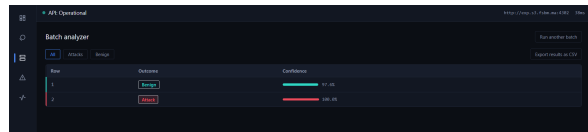
(a) Tableau de bord (Dashboard)



(b) Inspecteur — échantillon Benign



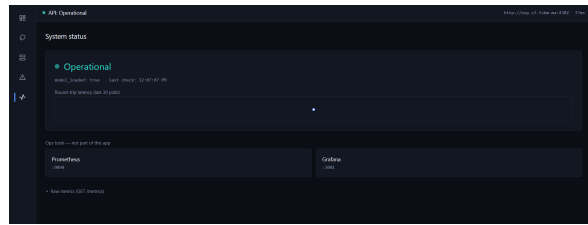
(c) Inspecteur — échantillon Attack



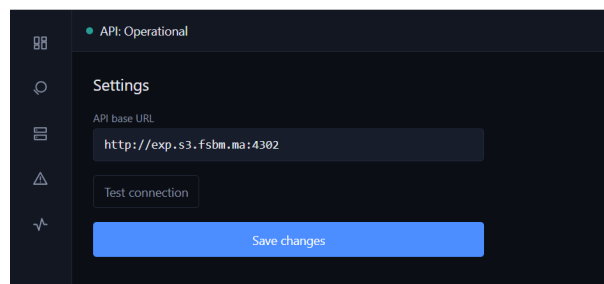
(d) Analyse par lots (Batch)



(e) Alertes en temps réel



(f) État du système

Figure 7.5: Pages principales du frontend React de la plateforme MLSecOps.**Figure 7.6:** Page de configuration (Settings) — URL de base de l'API.

Ces captures illustrent les six pages du frontend décrites au Tableau 2.2 : la vue d'ensemble du tableau de bord, l'inspection manuelle d'un flux unique (avec un exemple bénin et un exemple d'attaque), l'analyse par lots via téléchargement CSV, le journal des alertes en temps réel, l'état de santé du système, et la configuration de l'URL de l'API.

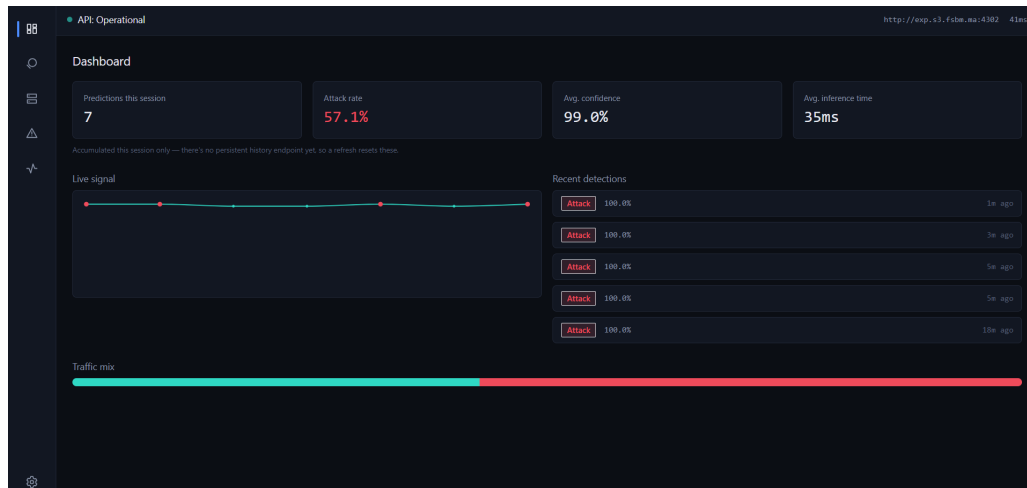


Figure 7.7: Interface frontend de la plateforme MLSecOps (Section Hero).

7.4 Architecture de Surveillance (Monitoring)

La pile de surveillance (monitoring stack) se compose de trois couches :

1. **Instrumentation** (Niveau Application) : L'application FastAPI expose trois métriques Prometheus via le point de terminaison `/metrics` : le nombre de prédictions, le nombre d'anomalies et la latence d'inférence.
2. **Collecte** (Prometheus) : Effectue une collecte (scrape) du point de terminaison des métriques FastAPI toutes les 15 secondes et stocke les données de séries chronologiques dans son volume TSDB.
3. **Visualisation** (Grafana) : Fournit des tableaux de bord (dashboards) avec Prometheus comme source de données, permettant :
 - La surveillance du taux de prédiction en temps réel
 - L'analyse des tendances de détection des anomalies
 - Le suivi des centiles de latence d'inférence (p50, p95, p99)
 - La configuration d'alertes en cas de dépassement de seuil

Cette architecture à trois niveaux (tiers) suit les meilleures pratiques de l'industrie pour l'observabilité dans les systèmes ML, où la surveillance des performances du modèle est aussi critique que la surveillance de l'infrastructure.

Chapter 8

Gestion de Projet : Méthodologie Agile avec Jira

8.1 Organisation Agile Scrum

Le développement de la plateforme MLSecOps a été piloté selon une méthodologie **Scrum**, outillée avec **Jira** pour la gestion du backlog produit, la planification des sprints et le suivi de l'avancement. Le projet a été découpé en **quatre sprints d'une semaine** (préfixe **SCRUM-*** pour l'ensemble des tickets), chacun aligné sur une couche fonctionnelle de l'architecture décrite dans les chapitres précédents :

- **Sprint 1** — Cadrage, étalonnage et ingestion (5–11 août).
- **Sprint 2** — Qualité des données et streaming (12–18 août).
- **Sprint 3** — Modélisation ML et suivi MLflow (19–25 août).
- **Sprint 4** — API, CI/CD et déploiement (26 août–1^{er} septembre).

Cette structuration en sprints a permis une livraison incrémentale et démontrable de chaque brique du pipeline (ingestion, qualité des données, entraînement, service d'inférence, orchestration, surveillance), tout en conservant une visibilité continue sur l'avancement via le tableau de bord Jira.

8.1.1 Sprint 1 — Cadrage, Étalonnage & Ingestion

Ce premier sprint a posé les fondations du projet : définition des objectifs et de la problématique de détection d'anomalies réseau, mise en place du Product Backlog initial, construction du premier pipeline d'ingestion et automatisation du téléchargement du jeu de données CICIDS2017.



Figure 8.1: Tableau Jira du Sprint 1 (*Cadrage, Étal & Ingest*, 5–11 août) : cadrage de la problématique, des objectifs et de la Data Strategy, mise en place du Product Backlog initial et premières tâches d'ingestion du jeu de données CICIDS2017.

8.1.2 Sprint 2 — Qualité des Données & Streaming

Le deuxième sprint s’est concentré sur le nettoyage et la normalisation des flux réseau en temps réel, la transformation des données via dbt, l’orchestration de l’ingestion en pipeline, ainsi que la validation automatisée de la qualité des enregistrements (le framework de 23 règles de qualité présenté au Chapitre 3 y trouve son origine).

S2: Data Quality & Orchest. 12 Aug – 18 Aug (7 work items)			
SCRUM-21	Nettoyer et normaliser les enregistrements de flux en temps réel	Pipeline DataOps	Idea
SCRUM-22	Transformer les données nettoyées avec dbt sur un entrepôt DuckDB	Pipeline DataOps	Idea
SCRUM-23	Orchestrer l'ensemble du pipeline avec Dagster (assets, capteurs, planification)	Pipeline DataOps	Idea
SCRUM-24	Valider automatiquement la qualité des CSV bruts avant ingestion	Qualité des Données	Idea
SCRUM-25	Valider la qualité des enregistrements nettoyés et de la sortie d'inférence	Qualité des Données	Idea
SCRUM-26	Définir des Data Contracts entre les étapes du pipeline	Qualité des Données	Idea
SCRUM-27	Documenter les jeux de données du projet	Qualité des Données	Idea

Figure 8.2: Tableau Jira du Sprint 2 (*Data Quality & Ingest*, 12–18 août) : nettoyage et normalisation des flux, transformations dbt, et mise en place des portes de qualité des données.

8.1.3 Sprint 3 — Modélisation ML & MLflow

Le troisième sprint a porté sur la sélection des caractéristiques pertinentes (en excluant les colonnes sources de fuite de données), ainsi que sur la mise en place du suivi des expériences et de la sauvegarde des modèles entraînés via MLflow.

S3: Modélisation ML & MLflow 19 Aug – 25 Aug (3 work items)			
SCRUM-28	Sélectionner les caractéristiques pertinentes et prévenir la fuite de données	Modélisation ML & MLflow	Idea
SCRUM-29	Entraîner un modèle XGBoost robuste au déséquilibre des classes	Modélisation ML & MLflow	Idea
SCRUM-30	Suivre toutes les expériences et versionner les modèles avec MLflow	Modélisation ML & MLflow	Idea

Figure 8.3: Tableau Jira du Sprint 3 (*Modélisation ML & MLflow*, 19–25 août) : sélection des caractéristiques, prévention de la fuite de données et suivi des expériences avec MLflow.

8.1.4 Sprint 4 — API, CI/CD & Déploiement

Le dernier sprint a livré la couche de service : exposition du modèle via une API REST FastAPI (prédiction unitaire et par lots), sécurisation des accès, conteneurisation complète de la plateforme et automatisation de la construction des images via le pipeline CI/CD, jusqu’au déploiement de l’ensemble des services.

S4: API, CI/CD & Déploiement 26 Aug – 1 Sep (5 work items)			
SCRUM-31	Exposer le modèle via une API REST FastAPI (prédiction unique et par lots)	Déploiement, CI/CD	Idea
SCRUM-32	Surveiller la santé et les performances du modèle en production	Déploiement, CI/CD	Idea
SCRUM-95	Fournir un tableau de bord pour visualiser les menaces et interagir avec le modèle	Déploiement, CI/CD	Idea
SCRUM-99	Conteneuriser tous les services et automatiser les tests/build avec CI	Déploiement, CI/CD	Idea
SCRUM-104	Déployer la plateforme sur l'infrastructure partagée Komodo	Déploiement, CI/CD	Idea

Figure 8.4: Tableau Jira du Sprint 4 (*API, CI/CD & Deployment*, 26 août–1^{er} septembre) : exposition du modèle via FastAPI, sécurisation, conteneurisation et automatisation CI/CD.

8.2 Backlog Produit et Traçabilité

Au-delà des quatre sprints planifiés, un **backlog produit** de plus de 70 éléments a été maintenu dans Jira tout au long du projet, regroupant à la fois des user stories fonctionnelles et des tickets de correction de bogues (*bug tracking*). Ce backlog a notamment permis de tracer plusieurs problèmes rencontrés en cours de développement, tels que :

- Des échecs de téléversement de fichiers CSV liés à des valeurs invalides (*Infinity/NaN*) dans le jeu de données CICIDS2017.
- Des arrêts du pipeline Dagster déclenchés par les portes de qualité des données sur certaines colonnes de flux.
- Des désaccords de schéma de colonnes entre les données d'entraînement et les données servies au moment de l'inférence.
- Des scores de confiance anormalement faibles pour la classe *Attack*, ayant motivé une revue des poids d'échantillonnage du modèle.
- Des durcissements de sécurité supplémentaires pour le pipeline CI/CD.

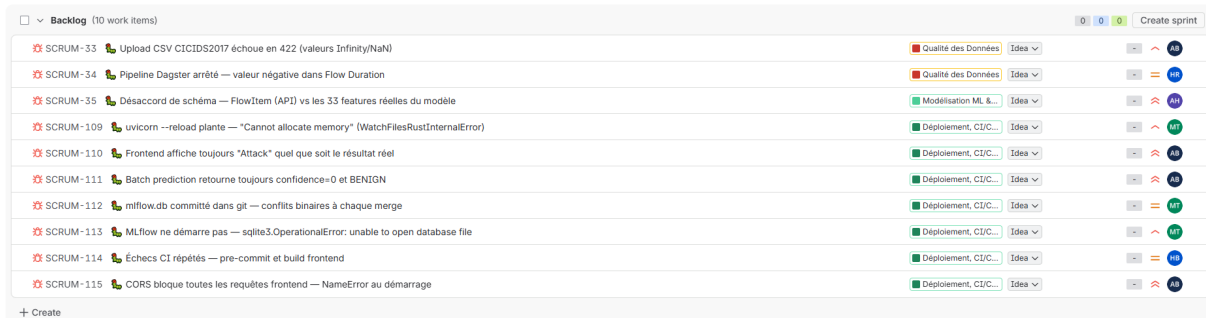


Figure 8.5: Extrait du backlog produit Jira (plus de 70 éléments), incluant les tickets de fonctionnalités et de correction de bogues répartis sur l'ensemble des sprints du projet.

Ce suivi rigoureux via Jira a permis d'assurer une traçabilité complète entre les besoins fonctionnels, les incidents techniques rencontrés et les livrables finaux documentés dans ce rapport, conformément aux bonnes pratiques de gestion de projet en contexte MLOps.

Chapter 9

Conclusion & Travaux Futurs

9.1 Résumé des Contributions

Ce projet a conçu, mis en œuvre et documenté avec succès une **plateforme MLSecOps** complète pour la détection d'anomalies réseau. La plateforme démontre l'intégration des opérations de données (DataOps), des opérations d'apprentissage automatique (MLOps) et des opérations de sécurité au sein d'un seul système entièrement conteneurisé. Les principales contributions sont les suivantes :

1. **Pipeline de Bout en Bout** : Un pipeline de données complet, de l'ingestion CSV au flux de données (Kafka/Redpanda), nettoyage, validation de la qualité, ingénierie des caractéristiques, entraînement du modèle, inférence et service d'API.
2. **Cadre Robuste de Qualité des Données** : Un framework de validation en trois étapes de 831 lignes imposant 23 règles de qualité à travers les dimensions d'exhaustivité, de validité et d'intégrité, avec des sémantiques d'arrêt du pipeline.
3. **Pipeline ML de Qualité Production** : Entraînement XGBoost avec validation croisée stratifiée, pondération des échantillons atténuée par la racine carrée, prévention complète des fuites de données et suivi intégral des expériences MLflow.
4. **Architecture Sécurisée dès la Conception (Secure-by-Design)** : Crochets de pré-validation, CI/CD avec quatre portes de qualité, exécution de conteneurs non-root, accès API restreint par CORS, validation des entrées Pydantic et suppression explicite des colonnes sensibles.
5. **Frontend Moderne** : Une SPA (Single Page Application) React + TypeScript de six pages avec des tableaux de bord en temps réel, téléchargement CSV par lots, inspection de flux individuel et surveillance de l'état du système.
6. **Orchestration Complète** : Orchestration d'actifs basée sur Dagster avec des capteurs de système de fichiers, des planifications horaires, des portes de qualité et de riches annotations de métadonnées.
7. **Infrastructure Observable** : Surveillance Prometheus + Grafana avec des métriques ML personnalisées (nombre de prédictions, taux d'anomalies, latence d'inférence).
8. **Couverture de Test Étendue** : 537 lignes de tests de qualité des données basés sur pytest couvrant les trois étapes de validation avec des cas de test positifs, négatifs

et aux limites.

9.2 Limites

Malgré son exhaustivité, la plateforme présente plusieurs limites :

- **Déploiement mono-nœud** : L'architecture Docker Compose ne prend pas en charge la mise à l'échelle horizontale. Un déploiement de production nécessiterait Kubernetes.
- **Hyperparamètres statiques** : Les hyperparamètres XGBoost sont codés en dur. Le réglage automatisé des hyperparamètres (Optuna, Ray Tune) améliorerait les performances du modèle.
- **Pas de détection de dérive (drift) des données** : La plateforme ne surveille pas le changement de distribution entre les données d'entraînement et les données servies.
- **Robustesse adversarielle limitée** : Le modèle n'a pas été testé contre des exemples adversariels (attaques d'évasion).
- **Pas de test A/B de modèle** : La couche de service ne prend pas en charge la répartition du trafic entre les versions du modèle.
- **Les modèles dbt sont échafaudés** : La couche de transformation dbt contient un modèle de mise en scène (staging) de démonstration plutôt que des transformations de production complètes.

9.3 Travaux Futurs

Plusieurs directions de développement futur renforceraient la plateforme :

1. **Déploiement Kubernetes** : Migrer de Docker Compose vers Kubernetes (avec des charts Helm) pour l'évolutivité en production, permettant la mise à l'échelle automatique du service d'inférence FastAPI en fonction des métriques Prometheus.
2. **Détection de la Dérive des Données** : Intégrer Evidently AI ou Great Expectations pour une surveillance continue de la dérive de distribution des caractéristiques entre les données d'entraînement et les données servies.
3. **Test de Robustesse Adversarielle** : Mettre en œuvre des simulations d'attaques d'évasion (FGSM, PGD) contre le modèle XGBoost et évaluer les techniques de défense (entraînement adversarial, masquage du gradient d'entrée).
4. **Optimisation Automatisée des Hyperparamètres** : Intégrer Optuna avec le suivi MLflow pour une recherche d'hyperparamètres bayésienne, consignée en tant qu'exécutions MLflow imbriquées.
5. **Test A/B de Modèle** : Mettre en œuvre la répartition du trafic dans la couche FastAPI en utilisant des drapeaux de fonctionnalités (feature flags), avec des tests de signification statistique pour la comparaison de modèles.

6. **Modèles d'Apprentissage Profond (Deep Learning)** : Évaluer des modèles de séquences basés sur LSTM/Transformer pour la détection de modèles temporels dans les flux de trafic réseau.
7. **Pipeline de Transformation dbt Complet** : Étendre la couche dbt avec des modèles mart, une matérialisation incrémentale et une intégration avec le pipeline d'exportation Parquet.
8. **Sécurité de la Chaîne d'Approvisionnement** : Intégrer Dependabot/Renovate pour l'analyse automatisée des vulnérabilités des dépendances et la génération de SBOM (Software Bill of Materials).
9. **Apprentissage Fédéré (Federated Learning)** : Explorer l'entraînement fédéré à travers des capteurs réseau distribués, permettant l'entraînement du modèle sans centraliser les données de trafic sensibles.

9.4 Remarques Finales

La plateforme MLSecOps démontre que la sécurité, l'excellence opérationnelle et l'apprentissage automatique peuvent coexister au sein d'une architecture unifiée. En traitant la qualité des données, le suivi des expériences, la sécurité des conteneurs et la qualité du code comme des préoccupations de premier ordre — plutôt que comme des réflexions après coup — la plateforme fournit une base sur laquelle des systèmes ML de niveau production et soucieux de la sécurité peuvent être construits. Nous pensons que ce travail contribue de manière significative au corpus croissant de recherche et de pratique à l'intersection du MLOps et de la cybersécurité.

Bibliography

- [1] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, “Hidden Technical Debt in Machine Learning Systems,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.
- [2] I. Sharafaldin, A. Habibi Lashkari, and A. A. Ghorbani, “Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization,” *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, 2018.
- [3] M. Zaharia, A. Chen, A. Davidson, A. Ghodsi, S. A. Hong, A. Konwinski, S. Murching, T. Nykodym, P. Ogilvie, M. Parkhe, et al., “Accelerating the Machine Learning Lifecycle with MLflow,” *IEEE Data Engineering Bulletin*, vol. 41, no. 4, 2018.
- [4] N. Schrock et al., “Dagster: An Open-Source Data Orchestrator for Machine Learning, Analytics, and ETL,” Elementl, 2019. [Online]. Available: <https://dagster.io>
- [5] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.
- [6] S. Ramirez, “FastAPI: Modern, Fast (High-Performance), Web Framework for Building APIs with Python 3.7+,” 2019. [Online]. Available: <https://fastapi.tiangolo.com>
- [7] B. Rabenstein and J. Volz, “Prometheus: A Next-Generation Monitoring System,” *login.*, vol. 40, no. 4, 2015.
- [8] D. Merkel, “Docker: Lightweight Linux Containers for Consistent Development and Deployment,” *Linux Journal*, vol. 2014, no. 239, 2014.
- [9] Fishtown Analytics, “dbt: Data Build Tool — Analytics Engineering Workflow,” 2016. [Online]. Available: <https://www.getdbt.com>
- [10] Redpanda Data, “Redpanda: A Kafka-Compatible Streaming Platform,” 2021. [Online]. Available: <https://redpanda.com>
- [11] A. Kumar, S. Saha, and R. Gupta, “MLSecOps: Integrating Security into Machine Learning Operations,” *IEEE Security & Privacy*, vol. 21, no. 3, 2023.
- [12] S. Amershi, A. Begel, C. Bird, R. DeLine, H. Gall, E. Kamar, N. Nagappan, B. Nushi, and T. Zimmermann, “Software Engineering for Machine Learning: A Case Study,” *Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2019.